Контрол на версиите на проекта. Git и Git Hub.

Контрол на версиите (version control)

Система за **контрол на версиите** се ползва за записване на промените по даден файл или колекция от файлове. Тя позволява да се запази история на промените, да се върне предишна версия, да се добави описание на промените (като например защо и на какво е извършена промяната и други). Когато по един проект с множество файлове работят много хора, едно от нещата, което би било трудно без такава система, е да се съобразяват промените по различните части на проекта от всички участници. Системата за контрол на версиите се развива през годините, като в началото ѝ стои **локалната система за контрол на версиите**, при която файловете са били на локалния компютър на разработчика и само той е работил с тях.

Контрол на версиите (version control)

След това се появява **централизирана система за контрол на версиите** (CVS, Subversion), която позволява много потребители да работят по един проект. Проектът се намира на един централен компютър, с който те осъществяват връзка.

Накрая се появява **децентрализирана система за контрол на версиите** (Darcs, Bazaar, Git, Mercurial, Fossil, Veracity), която решава най-големия недостатък на другите две: в случай на повреда в централния или локалния компютър, всички данни на проекта могат да бъдат загубени, тъй като се намират на едно физическо място. При децентрализираната система за контрол на версиите проектът се сваля и копира локално на компютъра на всеки участващ потребител, като така се създава бекъп. В случай на необходимост, проектът може да се възстанови от локалното копие на някой от потребителите.

Какво е Git?

Git е една от най-известните и използвани децентрализирани системи за контрол на версиите на файловете. С Git много потребители могат да работят едновременно по един и същ файл, без това да доведе до бъркотия и хаос в съдържанието на файла.

Git е създаден и разработен през април 2005г. от Линус Торвалдс (човекът стоящ зад Linux ядрото). Торвалдс създава Git, за да го използва за разработката на неговия проект Linux ядро. От тогава до днес Git се развива и подобрява от множество потребители. Включително самият проект на Git използва тази система, като много разработчици извършват промени в кода му (C, Shell).

Какво е Git?

Git е създаден с идеята да е бърз, с прост дизайн, стабилна поддръжка на **неправолинейна разработка – хиляди паралелни разклонения (branches), напълно децентрализирана организация** и възможност да поддържа ефикасно **големи проекти** (като Linux ядрото).

Част от проектите, които използват Git са Linux Kernel, KVM, Bacula, CakePHP, Drupal, FFmpeg, GNOME, phpMyAdmin, Ruby on Rails и много други. Сред проектите, които ползват Mercurial са Dovecot, nginx, OpenOffice, Python, rpm, Vim и други.

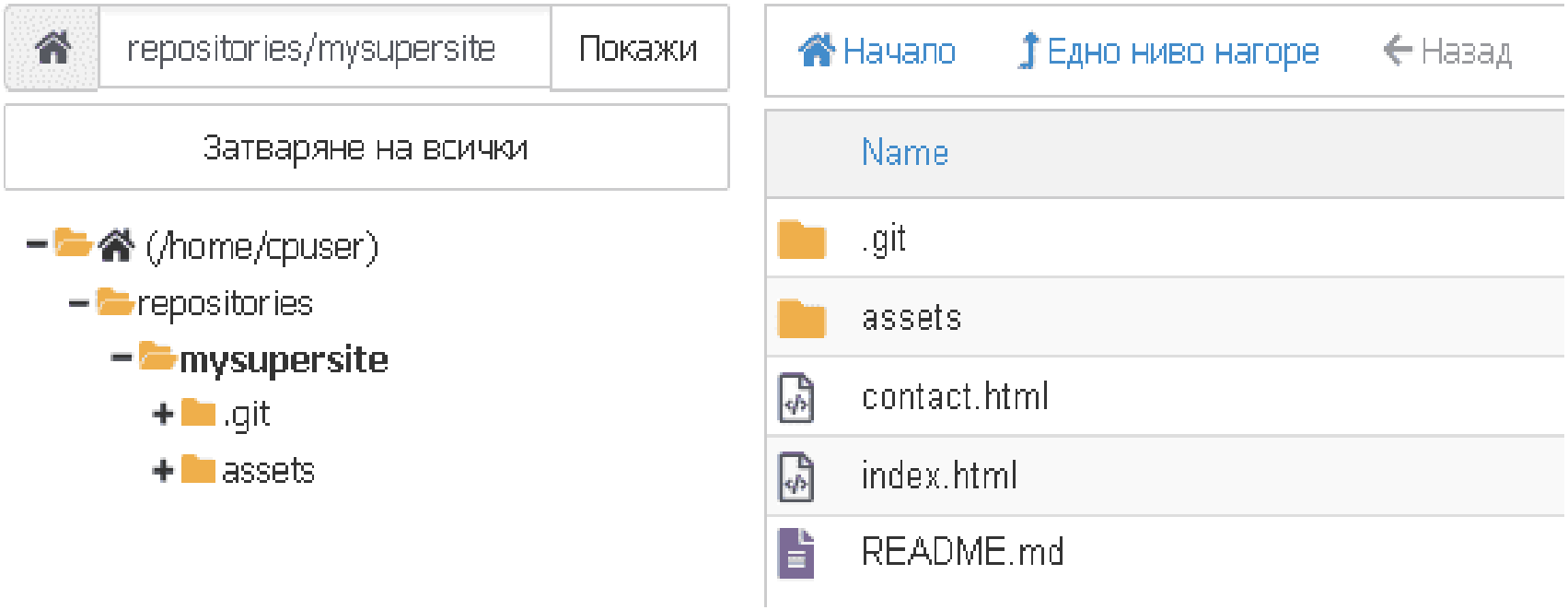
Какви са ползите от Git и за какво се използва?

Чрез Git е възможно един или много разработчици да прилагат промени в кода на файловете на един проект, по начин, чрез който се **избягват несъответствия и конфликти** от промените.

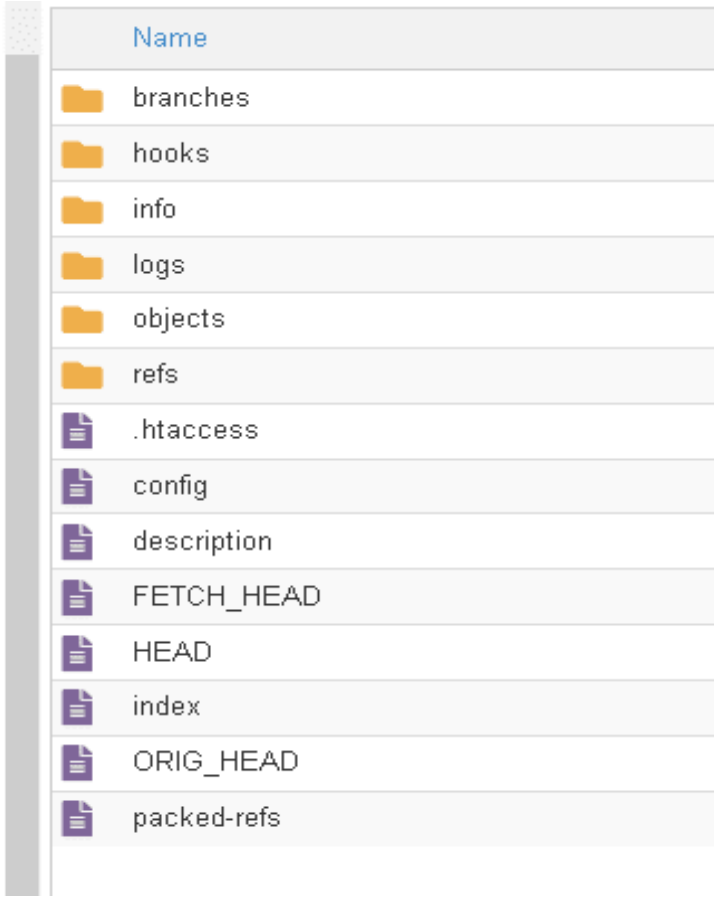
Git пази **история** за извършените промени, какво точно в кода е променено, от кого и кога и прави възможно **връщането към предишни версии** лесно.

Git работи с **хранилище (repository)**, което представлява директория, в която се съдържат файловете на проекта и една специална системна директория с име **.git.**

Директорията на хранилището, съдържаща файловете и системната директория .git.



В директорията .git се съхранява информация за проекта, промените по клоновете, файловете и кода в тях.



Git

С Git може да се създават разклонения (клон, branch) на основната линия на разработка. На тези клонове може да се извършват допълнителни разработки, които в краен етап да се слеят с основния клон (най-често наречен master).

Разработката и промяната на кода във файловете на проекта може да се извършва **отдалечено**, например на потребителското Ви устройство, а промените да се качват с Git в създаденото **cPanel хранилище**. Файловете в това хранилище може да се прехвърлят (Deploy) ръчно или автоматично към директорията на реалния сайт (например в public_html).

Друг вариант е, разработката да се случва **в отдалечено хранилище** като **GitHub**, а в cPanel само да се издърпват промените чрез опцията Pull и Deploy в Git Version Control.

Git

Разработчиците с опит, които **пишат код (*кодят*),** знаят защо и за какво използват Git. А учащите се разработчици може вече да са разбрали, че Git е един от най-известните и използвани инструменти при разработката на проект. И по-важното, че с Git могат да контролират версиите на кода, който създават.

Ако не пишете код и нямате намерение да го правите, но създавате и работите с текст, тогава ще Ви е интересно да знаете, че **Git работи и следи промените в съдържанието именно на текстови файлове**.

Git следи за промени по редовете в текстовия файл, независимо какво е съдържанието му – код, текст, числа, символи, всичко, което може да се запише в текстов формат.

Git

Кодът и текстът се съдържат в обикновени текстови файлове.

Поради това при Git и е възможно да се направи аналогия на „работата с код“ и „работата с обикновен текст“, например при редактиране на статия.

php-code.php

```
1   <?php
2   function twentytwenty_theme_support() {
3       // Add default posts and comments RSS f
4       add_theme_support( 'automatic-feed-link
5       // Custom background color.
6       add_theme_support( 'custom-backgroun
7           array('default-color' => 'f5efe0',)
8       );
9       // Set content-width.
10      global $content_width;
11      if ( ! isset( $content_width ) ) {$content_w
12  }
13  ?>
14
15
16
17
```

why-git.txt

```
1   # Какви са ползите от Git и за какво се изпол
2
3   Разработчиците с опит, които пишат код (_к
    какво използват [Git](https://help.superhostin
    учащите се разработчици може вече да са р
    най-известните и използвани инструменти п
    проект, в който може да участват повече хор
    могат да контролират версиите на кода, кой
4
5   ## Има няколко причини да избират Git
6
7   Ползите от Git са и причините много разраб
    Някои от ползите са резултат от сравняване
    за контрол на версиите, които са централиз
    [Subversion](https://subversion.apache.org/) и
    [CSV](https://en.wikipedia.org/wiki/Concurrent_
8
```

С какви функционалности е полезен Git?

Отзивите за Git от разработчиците са **положителни**.

Харесва се **скоростта**, с която работи, и **начина му за следене на промените по кода**.

Чувстват се спокойни от **сигурността** му, от гледна точка на **целостта на данните**, и естествения **бекъп на кода**, който може да „**върнат**“ при нужда.

С какви функционалности е полезен Git?

Поради всичките си ползи Git се използва от стотици хиляди разработчици. А Stack Overflow е един от най-известните портали за разработчици, в който споделят опит и намират помощ за казусите, с които се сблъскват всеки ден. Всяка година Stack Overflow прави проучване сред потребителите на портала, за да види кои инструменти и технологии използват. В проучване на **Stack Overflow** от 2018 г. са участвали над 74 хиляди потребители на портала. Почти **90%** от тях са отговорили, че инструментът за контрол на версиите на кода, който използват, е Git.

Някои от ползите са резултат от сравняването на Git с други системи за контрол на версиите, които са централизирани като **Subversion** и **CVS**. Например **по-бързата му работа** и **по-ефективният и лек подход** за разклоняване и сливане (**branching, merging**).

Съвместна работа и сътрудничество (**collaboration**)

Първата и основна полза от Git е присъща за всички системи за контрол на версиите. Това е **възможността за промяна на едни и същи файлове от повече от един човек**, **без** да се създават несъответствия и **конфликти** в съдържанието им. **Например** няколко човека извършват промяна в текста на една статия, като го подобряват и допълват. Но когато публикуват своите промени, те се добавят или сл**иват в статията, без да се създава конфликт в съдържанието ѝ**.

Ако го сравним със статиите, съвместната работа може да е – множество автори допринасят за подобряването на една и съща статия, но не като променят цялото ѝ съдържание наведнъж, а като всеки автор променя един определен абзац от нея. Може да го подобрява като изказ, да го разширява и допълва, да го видоизменя.

Съвместна работа и сътрудничество (**collaboration**)

Съвместната работа и споделеното допринасяне за разработката и подобряването на кода е в основата на Open Source идеята. Поради това Git се използва предимно от проекти с отворен код, към които безвъзмездно допринасят много хора. Ако сте голям фен например на WordPress като нас, не Ви ли е идвало наум нещо от сорта на „ех, ако можех да кодя, щях да направя нещо, да върна към WordPress обществото, което ми дава толкова много“?

Основното хранилище за кода на WordPress не използва Git, а Subversion, но това по никакъв начин не е попречило за огромния брой доброволци разработчици.

Групиране на промени преди публикуването им (**staging**)

Къмитът (**commit**) в Git е действието, с което в Git хранилището се публикуват промените по съдържанието на файловете. Ако промяната е в един файл, можем да го сравним с това да запишем промените, които сме направили в една статия. Къмитът е все едно кликване на бутона „Обновяване“ в WordPress редактора или „Запазване“ в MS Word.

Но преди да публикуваме промените, можем да ги подберем.

Git позволява публикуването на **промените да е обособено по тема**. Всеки къмит може да съдържа промени, които принадлежат на една тема, но се съдържат в отделни и различни файлове. За тази цел се използва **staging** пространството. Файловете с промените, които ще бъдат публикувани, се поставят в статус „staged“. Това е нещо като да запишем чернова на промените, които сме направили в няколко статии, преди да ги публикуваме. Например в няколко статии има препратка към друг сайт.

Групиране на промени преди публикуването им (**staging**)

Това е нещо като да запишем **чернова** на промените, които сме направили в няколко статии, преди да ги публикуваме. **Например** в няколко статии има препратка към друг сайт, чийто уеб адрес вече не е актуален. Променяме уеб адреса, **като го актуализираме** в тези отделни статии, и записваме „чернова“ на тези промени (черновата на промените в Git се нарича „добавяне в индекса“). Тази чернова с промени след това ще публикуваме с едно **действие – къмит (публикуване, запазване).** Така съдържанието на статиите ще бъде обновено наведнъж.

В представите на Git това публикуване е едно действие, на което можем да поставим и бележка с описание „Линкът към X е актуализиран“. Тоест причината за промените е една и това е „темата“ на къмита. Това прави процесът по разработка на съдържанието/кода кристално ясен. Можем след това да проследим къмитите и да видим какво, как, от кого, кога и защо е променено.

Групиране на промени преди публикуването им (**staging**)

Да се върнем към коденето. Ето **пример** за същото, но при разработката на приложение. Например, ако правим поправка на „bug#123“, което изисква множество промени по кода сред различни файлове, всички тези промени може да са в един къмит. Това ще позволи лесно проследяване на промените само за тази задача. Можем да опишем къмита като „fixed bug#123“. Ако срещнем в бъдеще подобен бъг, можем да разгледаме в детайли как е оправен стария. Друга съществена полза от публикуване на обособени промени е, че при нужда може да се върне предишното състояние на кода само за дадената разработка. Ако къмитът съдържа миксирани промени от различни теми, например за „компонент А“ и за „компонент Б“, при проблем в единия компонент, ще трябва се върнат промените и в двата компонента.

Обособените къмити са полезни не само за дадения разработчик и самия проект, но за колегите му, които работят по същия проект. **От къмита всеки може да разбере какво е променяно и защо**.

Без хаос и страх от експериментиране (**branching**)

Дори в най-малките проекти разработчиците често се налага да работят **върху множество компоненти** и задачи **паралелно** например „feature X“, „feature Y“, „bug#123“ и други. **Ако всички започнат да правят промени директно в основния код** на проекта, това ще доведе до **хаос**, трудно проследяване и връщане на отделните разработки.

Вместо това, всеки компонент и задача се разработва в **отделен клон – branch (разклонение) на основния код**. Разклонението в Git се базира на последната активна версия на основния код и позволява разработката да продължи в различна посока. Всеки от разработчиците **е клонирал Git хранилището на проекта локално на неговото устройство**. По този начин всеки един разполага с основния **master клон** и с пълната история на промените по целия код (и реално бекъп на целия код).

Без хаос и страх от експериментиране (**branching**)

От този основен клон се прави **разклонение, по което се извършват промените в кода.** Новият клон например „feature-x“ е един вид **прототип**. Докато един разработчик работи по „feature-x“, нищо и никой не е засегнат от неговия незавършен код. Ако се окаже, че този компонент всъщност не е нужен или пък след 10-тия къмит се установи, че пътят е грешен, разработчикът може да създаде лесно ново разклонение и да тръгне по друг път. Ако пренесем тази концепция за разработка в сферата на текстовото съдържание на статиите, това означава, че всеки автор ще има локално копие на основния клон и историята на съдържанието на всички статии в сайта. Съдържанието може да се намира в **текстови файлове**, като всяка статия е в отделен текстов файл.

Без хаос и страх от експериментиране (**branching**)

Според целите, всеки автор може да създава **обособен клон**, **по който да работи**, без това да засегне актуалното съдържание в публикуваната статия. Например създава разклонение „update-post-x”, в което започва да прави корекции по съдържанието на статията X. Стига донякъде, но вижда, че посоката в редакцията не е правилна, но създаденото дотук не е за изхвърляне. Затова създава ново разклонение и започва начисто, докато старото си стои запазено от Git, и ако пожелае, може да се върне към него.

Тук ще добавим и **процеса на Git за предотвратяване на конфликти в съдържанието.**

Без хаос и страх от експериментиране (**branching**)

Ако говорим за статии. **Например** авторът, който си е създал разклонението „update-post-x”, нанася корекции по съдържанието на статията X. През това време друг автор прави промени по същата статия и ги публикува в сайта. След като вторият автор е готов, пуска заявка за сливане (merge) на неговия клон (update-post-x) в основния клон (master). При опит за сливане на промените **Git проверява за конфликт във версиите**, като следи за припокриване на промени по отделните редове. Ако открие застъпване във версиите ще индикира към автора и ще му покаже кой точно ред и как се различава в неговата версия и във вече записаната от другия автор. Тъй като първият автор е направил промяна само по ред 128, а вторият автор е правил промяна само по първия ред в статията, Git слива промените без проблем. В реална обстановка и с правилен подход към тематични промени, възможността двама автори да изпълняват една и съща задача върху едно и също съдържание е минимална.

Без хаос и страх от експериментиране (**branching**)

Ако това се случи, най-малкото ще означава, че работният процес за управление на съдържанието не е ефективен. Например не е практика двама автори да имат една и съща задача – да ревизират съдържанието на една и съща статия, поне не докато се знае, че единият от тях е отговорник за задачата.

Git насърчава креативността и експериментирането в разработката, като премахва страха от това, че основната версия на проекта ще бъде засегната, докато опитваш нови неща.

Предоставя среда, в която можеш уверено да пробваш и развиваш бъдещ компонент към проекта.

Връщаш промени и отменяш грешки

С Git **можеш да връщаш направени промени лесно**. Можеш да „поправиш” последния къмит, защото си забравил да включиш нещо в него. Можеш да върнеш цял къмит, защото тази промяна всъщност вече не е нужна. Дори можеш **да възстановиш „изтрит“ къмит** с помощта на функционалността **reflog**, защото Git не изтрива почти нищо в реалния смисъл на думата. Дори изтриването е запис в историята на кода. Почти всяко действие в Git представлява добавяне на данни, не изтриване.

Възможността за отмяна (подобно на „**undo**“) дава на екипа куража да опитва нови идеи и концепции, без риск да счупят нещо, което в замяна отглежда култура на иновациите.

Повредило се е устройството или основното хранилище и целият проект е загубен? Няма страшно, всяко копие на хранилището, което се намира на устройствата на разработчиците служи като бекъп на цялото хранилище.

Ако се наложи, просто вземете локалното хранилище на някой от разработчиците и възстановете за минути. Някои разработчици използват Git дори само за това, да си имат бекъп на кода. Ако го сравним със статиите, връщането на промени е все едно да върнете предишната ревизия на съдържанието.

Други ползи от Git

- **Спестява време** – много е бърз и резгира почти мигновенно
- **Работите офлайн** – не е необходима мрежова връзка или със сървъра при работа локално на свой компютър от програмиста (редактора и др.)
- **Широко използван** - Популярността на Git се измерва предимно от използването му за проектите с **отворен код**. Git платформите като GitHub, GitLab и Bitbucket предоставят свободен начин за всеки да си направи Git хранилище. Git е интегриран в различни инструменти, които се използват от разработчиците: в интегрирани среди за разработка (Integrated Development Environment): Visual Studio, Eclipse, JetBrains и други.; в текстови редактори: Visual Studio Code, Sublime Text, Atom

Други ползи от Git

- **Git хранилище в хостинг акаунта** - Git е интегриран и в хостинг услугите. Можете да използвате Git за управление на версиите на Вашите текстови файлове или файлове с код в Git хранилище в хостинг акаунта Ви. С опцията Git Version Control в cPanel можете да създадете хранилище, в което да издърпвате (pull) или подавате (push) промени.
- **Публикуване на приложение** - Разработката на приложението може да използва уеб хранилище като GitHub или друго, а публикуването на промените в проекта да се случва в хостинг акаунта. Например създали сте проект, който искате да видите как работи в реална уеб среда – издърпвате промените в cPanel от отдалеченото хранилище и ги качвате (deploy) автоматично или ръчно в публичната директория на сайта.

Други ползи от Git

- **Статичен сайт за документация или портфолио** –

Разработчиците работят с код, но и често те може да са хората, които създават помощната документация за проекта или приложението си. Какво ще предпочете един човек, който е свикнал да борави с код и да не използва мишката – високата ефективност и бързина от този начин на работа или кликането на бутони? Прекрасният интерфейс на готовите CMS платформи не могат да го изкушат, защото ще трябва да жертва бързината и ефективността на работата си.

Това е причината много помощни документации към разработките да се създават и намират в статични сайтове. Разработчикът пише текста за помощната статия така, сякаш е код, в текстов файл. А **Git** е **идеалният инструмент за версиите на това съдържание, защото работи с текстови файлове**.

Съдържанието в текстовите файлове често е написано в някакъв лек маркиращ език като Markdown или reStructuredText. Системата на статичния сайт, която може да се намира в хостинг акаунта, преобразува съдържанието от текстовите файлове в HTML страници. Само един пример за такъв сайт: https://www.gatsbyjs.com/docs/.

Git сървър

За **Git сървър** можете да използвате cPanel от хостинг акаунт. През **Git Version Control** в cPanel можете да създадете **Git хранилище**, с което да се свързвате отдалечено от Вашето устройство.
В уеб може да се намерят и други услуги, предлагащи Git сървър като **GitHub**, Bitbucket и GitLab.
Най-прост Git сървър, без да се инсталира допълнително приложение, може да се направи само със **SSH достъп и подготвено Git хранилище на сървъра**. Такъв основен Git сървър може да се ползва за частен проект с малко на брой потребители. Съществуват и приложения за Git сървър, които могат да се инсталират на **VPS сървър**. Едно много използвано приложение, което много наподобява **GitHub е GitLab**. Такива приложения предлагат методи за управление, като контрол на достъпа, управление на множество Git хранилища, преглед през уеб и много други.

Git клиенти

Освен **стандартния начин за работа с Git – използване на команди през терминал**, съществуват и **графични Git клиенти**, които се инсталират локално на компютъра.

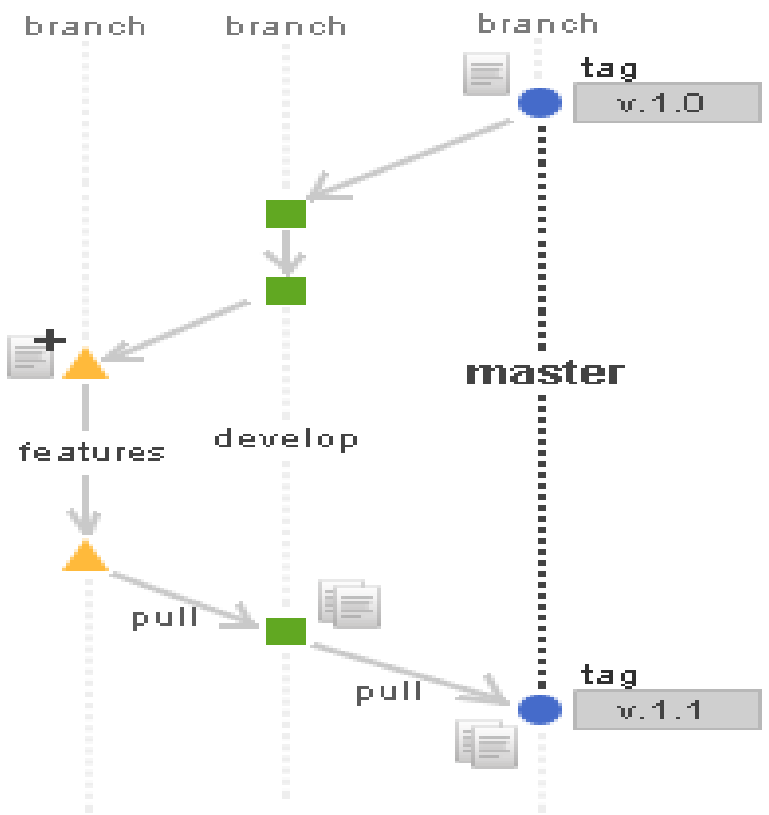
Текстовите Git клиенти се ползват през **шела на операционната система** и поради това можете да ги инсталирате и ползвате през терминала. За **Linux** базирани системи можете да инсталирате Git например с:

yum install git или apt-get install git

Windows потребителите могат да инсталират Git с **графичния** Git клиент на GitHub, който съдържа и Git **текстов** клиент (Git Shell).

Git клиент може да се ползва и през **акаунт на споделен хостинг**. Вижте какво е необходимо да направите, за да ползвате Git клиент в хостинг акаунта.

Основни понятия и команди в Git

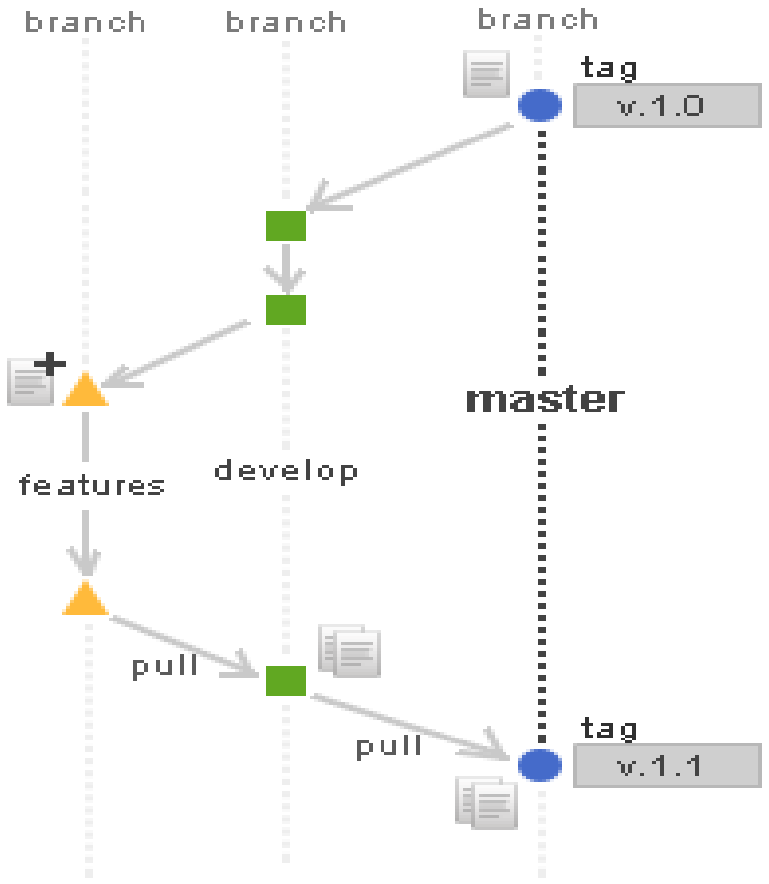


git repository (хранилище)

Работната директория на проекта.

Git хранилище можете да създадете от всяка директория, намираща се локално на Вашия компютър.

Или можете да клонирате хранилище намиращо се на друг компютър.

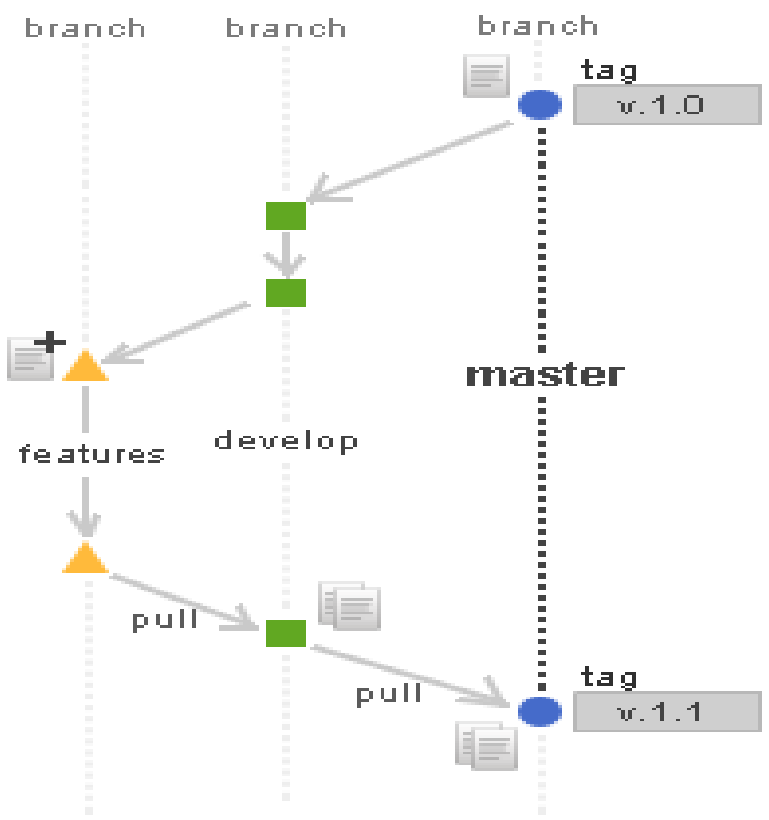


git branch (разклонение, клон, линия)

Git създава **дървовидна структура** на проекта с един основен клон, най-често наречен **master**.

Всичко, което се разработва и променя се извършва по други разклонения, които рано или късно се сливат с основния клон и така се ражда **нова версия на проекта**.

За разработката на различни аспекти от проекта може да се създадат специални разклонения, като **development**, **features**, **hotfixes** и други.



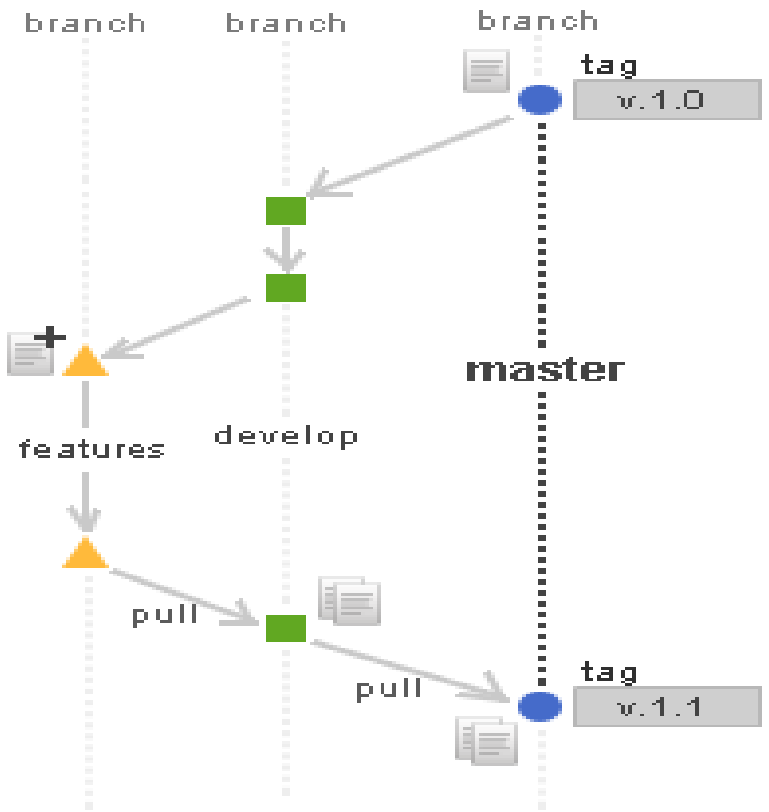
git tag

Всяка окончателно завършена и одобрена промяна се слива с **master** и представлява различна версия на проекта.

Промяната може да се надпише с различен таг, например започвайки с v. 1.0.0.

git add

С командата add добавяте промените в индекса на Git, които смятате, че са готови за публикуване (**commit**). Файлът, в който са промените, преминава в статус „**staged**“. Всички промени във файлове, които са в статус „готови“ (**staged**), може да се публикуват след това с commit.

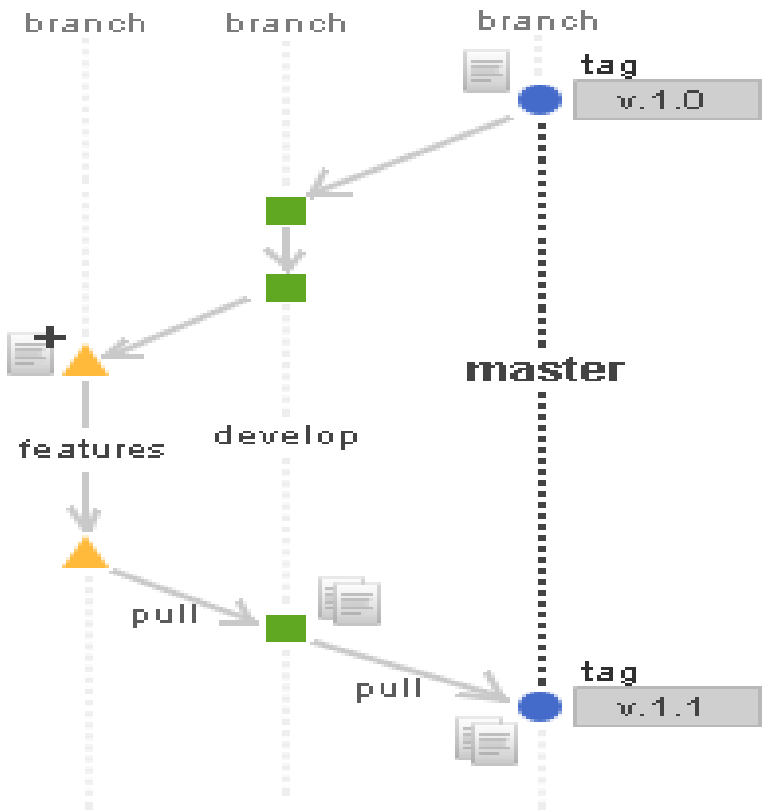


git commit

Публикуване на промените. Когато промените съдържанието на даден файл, намиращ се в директорията на проекта, ще трябва след това да направите **commit** заявка, за да публикувате тези промени. Ако използвате графичен Git клиент, Git ще Ви покаже, че е била извършена промяна и ще Ви попита, дали искате да направите commit (публикуване) на тази промяна.

git pull

Изтегляне на промени от едно хранилище/клон в друго хранилище/клон.



git push

Подаване на промените към отдалечено хранилище.

git merge

Сливане на промените в един клон или сливане на отделни клони. След като промените по файловете в даден клон (**branch**) са записани, можете да създадете pull заявка за сливане. Тази pull заявка ще е в статус изчакване за одобрение. След одобрение, посоченият клон ще бъде слят, използвайки git merge, с друг избран клон.

Създаване на локално Git хранилище

За да можете да създавате и управлявате **Git хранилища** на Вашето устройство, първо е нужно да инсталирате Git. От официалния сайт на Git свалете и инсталирайте официалното приложение за Git: https://git-scm.com/downloads (Windows). След като сте инсталирали Git на Вашия компютър, можете да създавате Git хранилище от всяка директория. В директорията може вече да има файлове. За да създадете Git хранилище, достъпете през Git шела избраната директория. И след това изпълнете командата: ***git init***

Създаване (init) на локално Git хранилище през Git Bash.

В директорията ще се появи поддиректория .git, в която се съдържат системните файлове на Git.

Създайте или поставете файл в директорията, например **file.txt** и изпълнете:

```
git status
```

Ще видите съобщението:

Untracked files: (use „git add …" to include in what will be committed)

Това означава, че Git открива този файл, но не е инструктиран да го следи (за промени). С командата **add** добавяте промените в индекса на Git. За да подготвите промените за публикуване, изпълнете командата:

```
git add file.txt
```

Публикуване (commit) на промените в хранилището.

И след това публикувайте промените с:

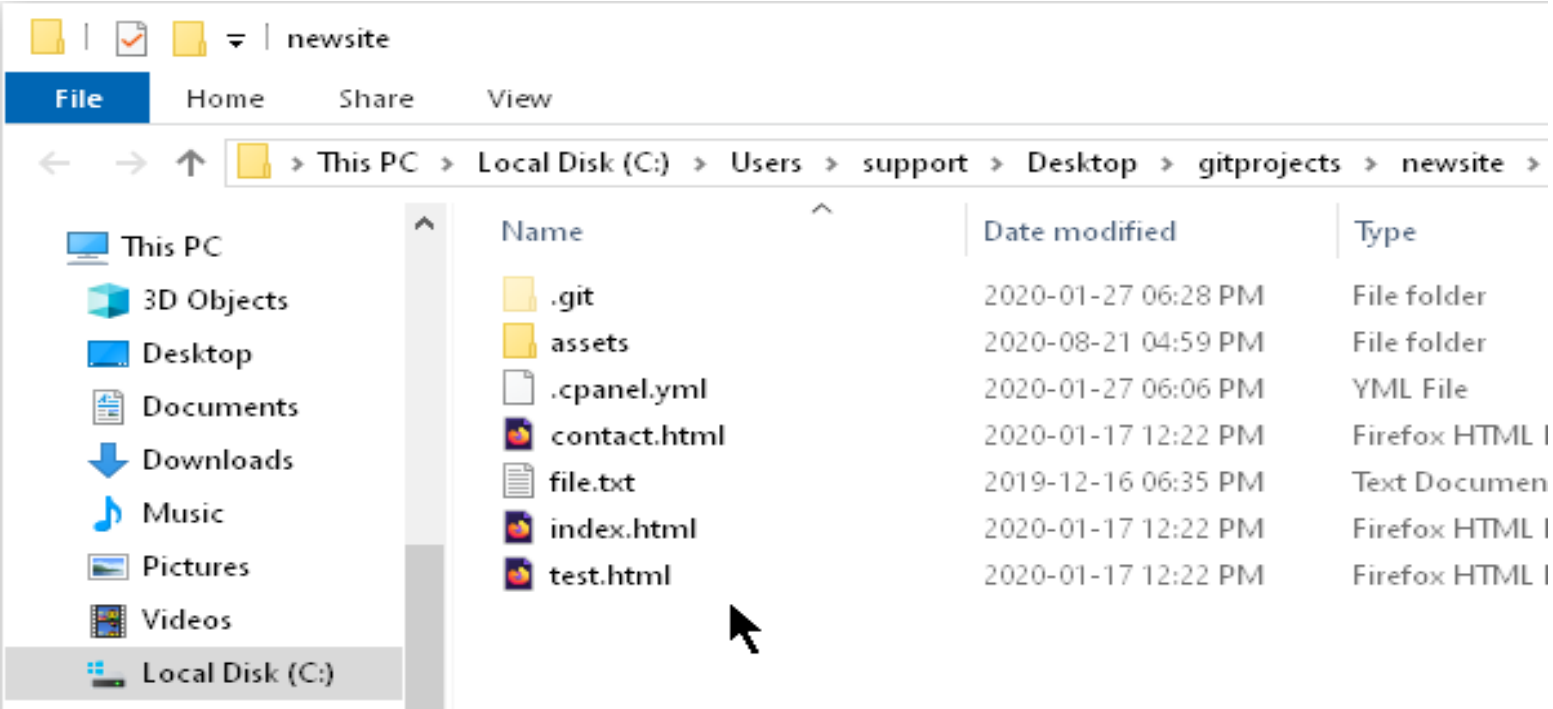
```
git commit -m „New file added“
```

```
support@support MINGW64 ~/Desktop/gitprojects/newsiterepo (master)
$ git commit -m "New file added"
[master (root-commit) 9efc8e8] New file added
 1 file changed, 1 insertion(+)
 create mode 100644 newfile.txt

support@support MINGW64 ~/Desktop/gitprojects/newsiterepo (master)
$
```

„New file added“ е бележка към къмита, съвсем кратко описание на промените. Когато преглеждате историята на промените, ще виждате тези бележки, поради това използвайте смислено описание на промените.

Директорията на Git хранилището в Windows Explorer.



Локално Git хранилище можете да създадете на Вашето устройство, **след като инсталирате Git на него**, например можете да инсталирате Git GUI/Bash на Windows. Освен на Вашето устройство, Git хранилище можете да създадете в хостинг акаунта през Git Version Control в cPanel или през SSH достъп.
Не всички проекти, ползващи Git, са еднакви, с еднакви разклонения и процес на разработка. Всеки проект може да има различни изисквания за използването на клони и процеса по обновяване и прилагане на промените. Преди да клонирате някой проект и да започнете да допринасяте за развитието му, проверете документацията и README.md файла за упътване.

Основни Git понятия и команди

Няколко основни команди и понятия в Git.

git status

С командата status можете да разберете **кой е активният клон (branch),** с който работите в момента. Също така тази команда е много важна, за да разберете **статуса на файловете в проекта**.

Git работи с няколко **статуса на файловете**:

- **Untracked** – файл, който не се следи от Git.
- **Unmodified** – файл, в който няма промени (непроменен).
- **Modified** – файл, в който има промяна (променен).
- **Staged** – файл с промени, които са готови за публикуване (готов).

След като даден файл е в статус **staged**, промените в него се публикуват с командата **commit**. Поставянето на файлове в статус готови (staged), се извършва с командата **add**.

Основни Git понятия и команди

git config

След като сте инсталирали Git на Вашето устройство, може да извършите няколко предварителни настройки с командата **config**.

Към командата **config** може да се използва един от трите **параметъра**: –**system, –global, –local**. Според използвания параметър, настройките се записват в различни файлове, намиращи на различно място.

При използване на „**system**“ настройките се записват във файл ***/etc/gitconfig*** и обхващат всички потребители на системата, в която работи Git. При използване на „global“ настройките се записват в ***~/.gitconfig или ~/.config/git/config*** и са валидни само за дадения потребител на системата. При използване на „local“ настройките се отнасят за даденото Git хранилище и се записват във файл config в директорията на хранилището.

Основни Git понятия и команди - Примери:

Задаване на потребител и имейл адрес. Тази настройка е важна да се извърши първа след инсталацията на Git, тъй като тези данни ще се използват при всеки **commit**.

git config --global user.name "IvanPetrov"

git config --global user.email email@mysupersite.com

Задаване на текстов редактор, който ще се използва от Git:

git config --global core.editor "vim"

Проверка на зададените настройки в конфигурацията на Git:

git config --list

git config --show-origin user.name

Основни Git понятия и команди

git init

С тази команда се създава **ново празно Git хранилище** или може да се инициализира наново вече създадено.

В директорията, в която е изпълнена командата, се създава поддиректорията **.git**, която съдържа системните файлове и директории на Git.

След инициализиране на хранилището, Git все още не следи нито един файл в директорията. Ако директорията е празна, трябва първо да добавите файлове в нея. Ако вече има файлове, може да укажете на Git да ги следи.

Основни Git понятия и команди - Примери:

За създаване на хранилище от вече съществуваща директория с файлове:

git init

Изпълнявате git init, докато се намирате в директорията.

git add .

Тази команда ще добави всички налични файлове в директорията към системата за контрол на версиите. Ако добавите нов файл в директорията може да го включите към Git с командата: „add filename“.

След като подготвите промените в хранилището, за да ги публикувате, изпълнете:

git commit -m "first commit"

С тази команда се записва състоянието на хранилището и извършените промени (добавянето на файловете към Git).

Основни Git понятия и команди

git clone

С командата clone можете да клонирате вече съществуващо хранилище, намиращо се на отдалечен сървър.

При клонирането се **изтеглят всички важни данни на хранилището**. В случай че оригиналното хранилище се повреди, от клонираното е възможно да се възстанови състоянието му, в което е било в момента на клонирането.

Примери:

Клониране на отдалечено cPanel хранилище:

git clone ssh://goodexam@goodexample.site:1022/home/goodexam/newsite

Клониране на отдалечено GitHub хранилище:

git clone git@github.com:myproject/mysupersite.git

Основни Git понятия и команди

git remote
Управление на връзките с отдалечените хранилища. С **remote можете да добавите отдалечено хранилище**, което ще има **съкратено име**, чрез което можете да се обръщате към него.
Примери:
Добавяне на отдалечено cPanel хранилище, под името origin:
git remote add origin ssh://goodexam@goodexample.site:1022/home/goodexam/newsiterepo
Ако локалното хранилище е празно, може да се изтегли състоянието на отдалеченото хранилище след това с **git fetch**.
За да видите всички добавени отдалечени хранилища, добавете атрибута **„-v“**:
git remote -v

Основни Git понятия и команди

За да видите всички добавени отдалечени хранилища, добавете атрибута **„-v“**:

git remote -v

```
support@support MINGW64 ~/Desktop/gitprojects/reponewsite (master)
$ git remote -v
origin  ssh://goodexam@goodexample.site:1022/home/goodexam/newsiterepo (fetch)
origin  ssh://goodexam@goodexample.site:1022/home/goodexam/newsiterepo (push)
p5      git@github.com:shosttest/mysupersite.git (fetch)
p5      git@github.com:shosttest/mysupersite.git (push)
pr3     https://shosttest@bitbucket.org/shosttest/testproject.git (fetch)
pr3     https://shosttest@bitbucket.org/shosttest/testproject.git (push)

support@support MINGW64 ~/Desktop/gitprojects/reponewsite (master)
$
```

Списък с добавени отдалечени хранилища.

Когато искате да извършите някакво действие с отдалеченото хранилище, можете да използвате краткото му име, вместо URL адреса му, например:

git fetch origin

Основни Git понятия и команди

git add

С командата add **добавяте промените в индекса на Git**, които смятате, че са готови за публикуване (commit). Файлът, в който са промените, преминава в статус „staged“. Всички промени във файлове, които са в статус „готови“ (staged), може да се публикуват след това с commit.

След добавяне на нов файл в директорията на хранилището, подготвяне на тази промяна за публикуване:

git add "newfile.txt"

След промяна в даден файл, поставяне в статус „готов“ (staged):

git add "filename"

С add се извършва **индексиране на файла в Git,** което означава, че този файл е коректен и без конфликти.

Основни Git понятия и команди

git commit

След като промените са готови (staged), с командата commit ги публикувате.

Примери:

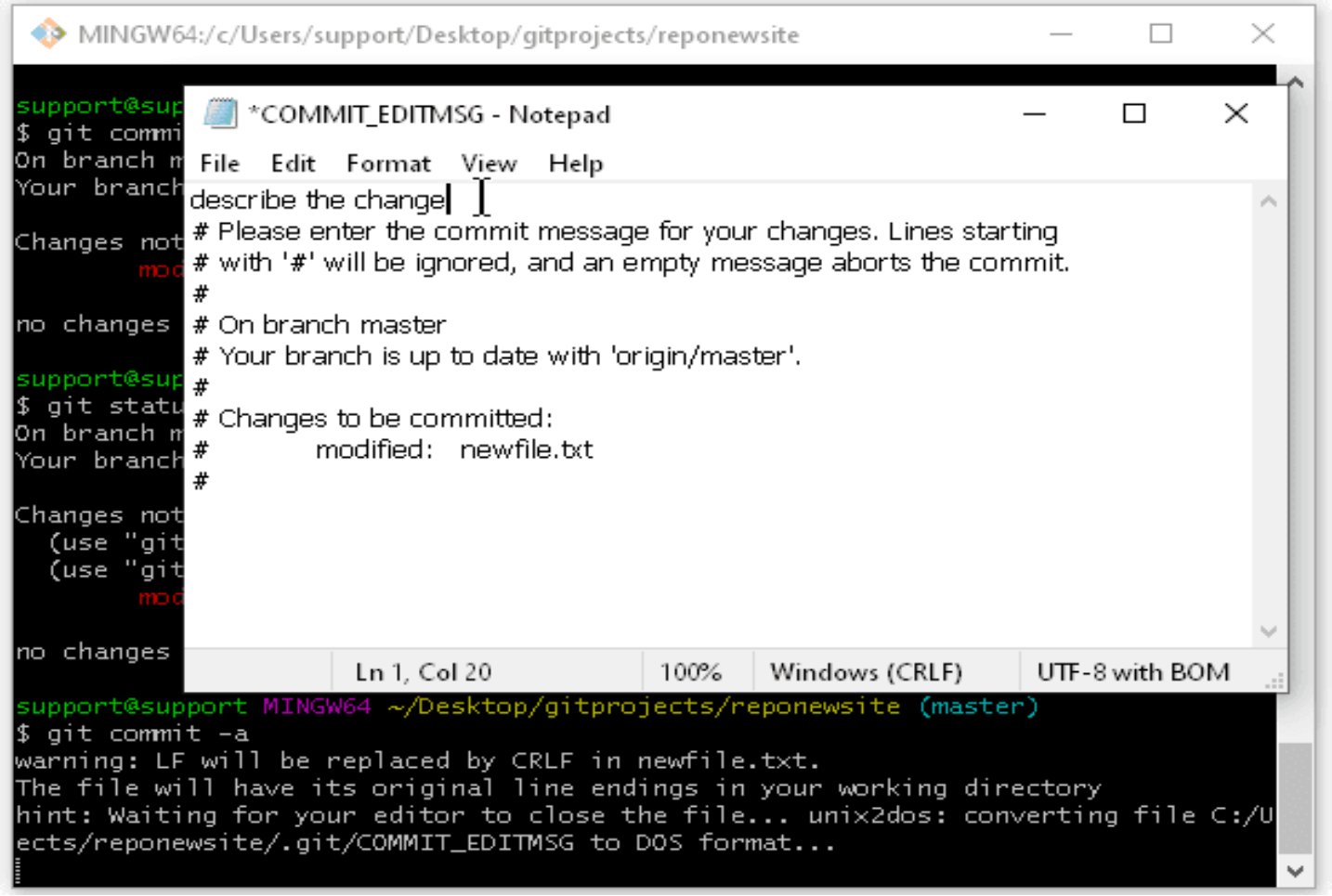
Публикуване на готови за публикуване (staged) промени:

git commit -m "describe the change"

Параметърът **„-m“** се използва, когато искате да въведете уточняващата бележка за промяната директно в конзолата. Ако не го въведете, Git ще отвори текстовия редактор по подразбиране, за да въведете описващата бележка към публикацията в него. Редакторът по подразбиране се задава в конфигурацията на Git например:

git config --global core.editor "notepad"

Основни Git понятия и команди



Git **зарежда редактора**, който е посочен в конфигурацията му.
След като запишете коментара в текстовия файл и го затворите, Git ще продължи и ще публикува промените.
За да прескочите етапа с подготвянето на промените, в който ги поставяте в статус staged (готов), може да добавите параметъра „**-a**“ към командата commit.
Двата параметъра „**-m**“ и „**-a**“ може да се комбинират в „**-am**„. Например:
git commit -am "note"

Основни Git понятия и команди

git branch

С командата branch се управляват клоновете в хранилището – създаване, преименуване и изтриване.

Например, за да създадете ново разклонение на разработката в хранилището (клон, branch), под името „nameofbranch“, използвайте:

git branch nameofbranch

За да превключвате между различни клони, използвайте:

git checkout nameofbranch

С тази команда се премества HEAD указателят към посочения клон.

Можете с една команда да създадете клон и да превключите към него:

git checkout -b nameofbranch

Превключването между клоновете променя файловете в работната директория.

За да изтриете клон, използвайте атрибута „-d“:

git branch -d nameofbranch

Основни Git понятия и команди

git branch

С командата branch се управляват клоновете в хранилището – създаване, преименуване и изтриване.

Например, за да създадете ново разклонение на разработката в хранилището (клон, branch), под името „nameofbranch“, използвайте:

git branch nameofbranch

За да превключвате между различни клони, използвайте:

git checkout nameofbranch

С тази команда се премества HEAD указателят към посочения клон.

Можете с една команда да създадете клон и да превключите към него:

git checkout -b nameofbranch

Основни Git понятия и команди

Превключването между клоновете променя файловете в работната директория.

За да изтриете клон, използвайте атрибута „-d“:

git branch -d nameofbranch

За да видите всички клони, използвайте само командата без атрибути:

git branch

```
support@support MINGW64 ~/Desktop/gitprojects/reponewsite (feature01)
$ git branch
  development
  fastfix
* feature01
  master
```

Списък с всички клони в проекта.

Клонът, посочен със **звездичка**, е **активният** в момента (HEAD указателят е преместен на него) и ако направите **commit**, промените ще се публикуват в този клон.

Основни Git понятия и команди

git pull

Тази команда се използва за автоматично изтегляне на промени от отдалеченото хранилище.

Когато текущият клон е настроен да проследява отдалечен клон (git remote add), с git pull можете да изтегляте данните от отдалечения клон.

Основни Git понятия и команди

git push

Командата push се използва, когато искате да публикувате промените в отдалеченото хранилище.

Качване на промените към отдалеченото хранилище:

git push origin master

Изпращане на промените в отдалечения клон.

```
support@support MINGW64 ~/Desktop/gitprojects/reponewsite (master)
$ git push -u origin master
stdin: is not a tty
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 241 bytes | 241.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
remote: Recieved update on checked-out branch, queueing deployment.
remote: ---
remote: apiversion: 3
remote: func: create
remote: module: VersionControlDeployment
remote: result:
remote:   data: ~
remote:   errors: ~
remote:   messages: ~
remote:   metadata: {}
remote:
remote:   status: 1
remote:   warnings: ~
To ssh://goodexample.site:1022/home/goodexam/newsiterepo
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.

support@support MINGW64 ~/Desktop/gitprojects/reponewsite (master)
$ |
```

Основни Git понятия и команди

git merge

Командата merge се използва, когато искате да слеете клон в друг.

Когато искате да направите сливане, трябва да сте превключили към клона, към който ще се извърши сливането. Например, за да слеете клона ***hotfix*** обратно към основния клон ***master***, първо превключвате към master: ***git checkout master***

И след това изпълнявате: ***git merge hotfix***

Сливане на клона fastfix в клона master.

В случай че сливането не се осъществи, поради конфликти в промените по кода в даден файл, ще е нужно първо да разрешите конфликта и след това да опитате отново със сливането.

```
support@support MINGW64 ~/Desktop/gitprojects/reponewsite (master)
$ git merge fastfix
Updating eb70fcf..744a7e5
Fast-forward
 newfile.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Основни Git понятия и команди

git log

С тази команда можете да извършвате преглед на историята на действията и да изведете списък с публикуванията (commits) в дадения клон. Към всеки commit се предоставя детайлна информация като името, имейла на автора, дата, бележката към публикуването и други. Към log командата може да се използват допълнителни атрибути, например за да ограничите резултатите до последните три commit-а, използвайте „-3“:

git log -3

История на извършените commit-и.

```
support@support MINGW64 ~/Desktop/gitprojects/reponewsite (master)
$ git log -3
commit 7f07a69fd8ab1693b45aaa0f4d731374d182b1a2 (HEAD -> master)
Author: Contributor01 <myemail@@mysupersite.com>
Date:   Wed Jan 15 19:48:45 2020 +0200

    note

commit be7144daf0dab582b71d3bc9ca4f8194d7cdd589
Author: Contributor01 <myemail@@mysupersite.com>
Date:   Wed Jan 15 19:23:32 2020 +0200

    describe the change

commit 593da327110565b0493bcbed9001e585e1f3fe12 (origin/master)
Author: Contributor01 <myemail@@mysupersite.com>
Date:   Thu Dec 12 21:10:53 2019 +0200

    New file added

support@support MINGW64 ~/Desktop/gitprojects/reponewsite (master)
$
```

Основни Git понятия и команди

Git псевдоними (Git aliases)

При изписване на команди, Git не ги допълва автоматично. Например ако започнете да изписвате командата „branch“, като изпишете първите две букви „br“ и натиснете Tab, командата няма да се завърши автоматично. Ако искате можете да създадете кратки варианти на командите (aliases).

За да създадете псевдоним за дадена команда, можете да поставите съответния ред в конфигурационния файл на Git, например:

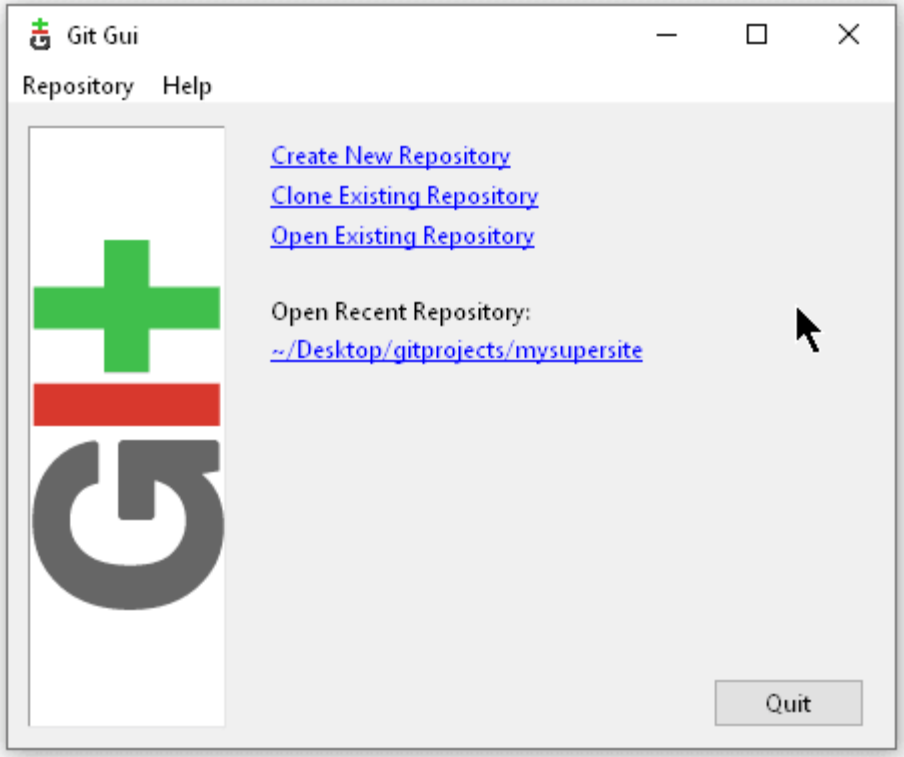
git config --global alias.br branch

С тази команда се добавя псевдоним „br“ за командата „branch“ в конфигурацията на Git. След това е нужно да изпишете само br, вместо цялата команда branch.

Основни Git понятия и команди

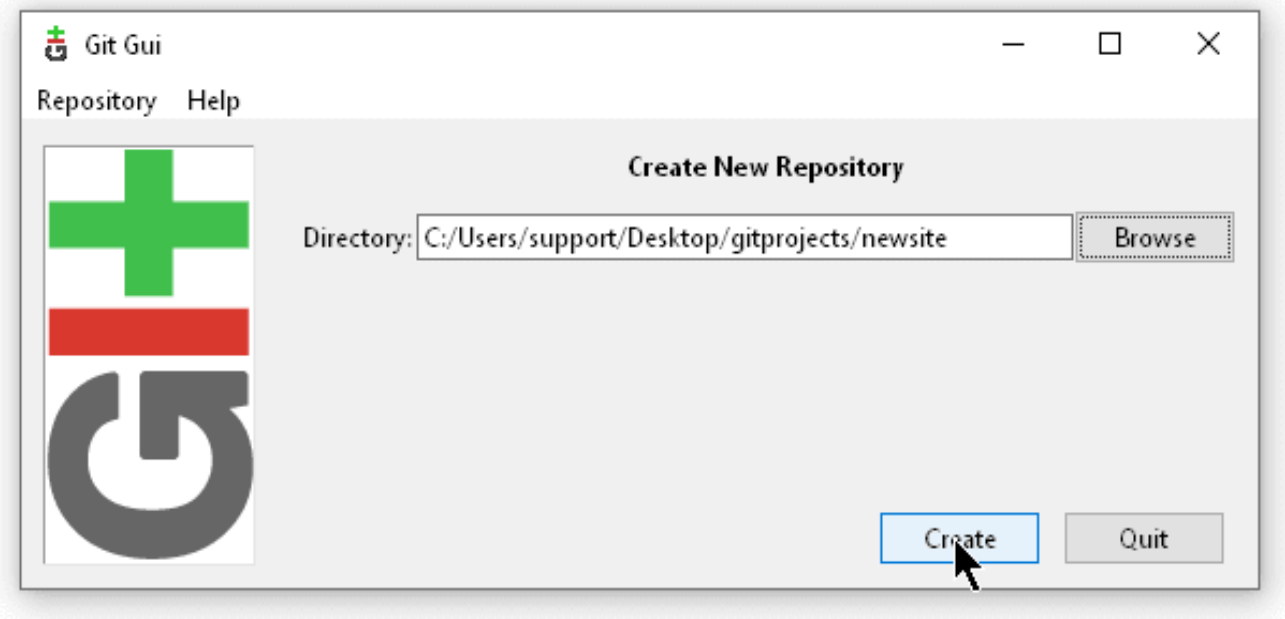
Създаване на ново локално Git хранилище през Git GUI

При стартиране на Git GUI на Windows се зарежда начален прозорец с няколко опции. За създаване на ново празно хранилище, изберете Create New Repository. Стартовият прозорец на Git GUI.



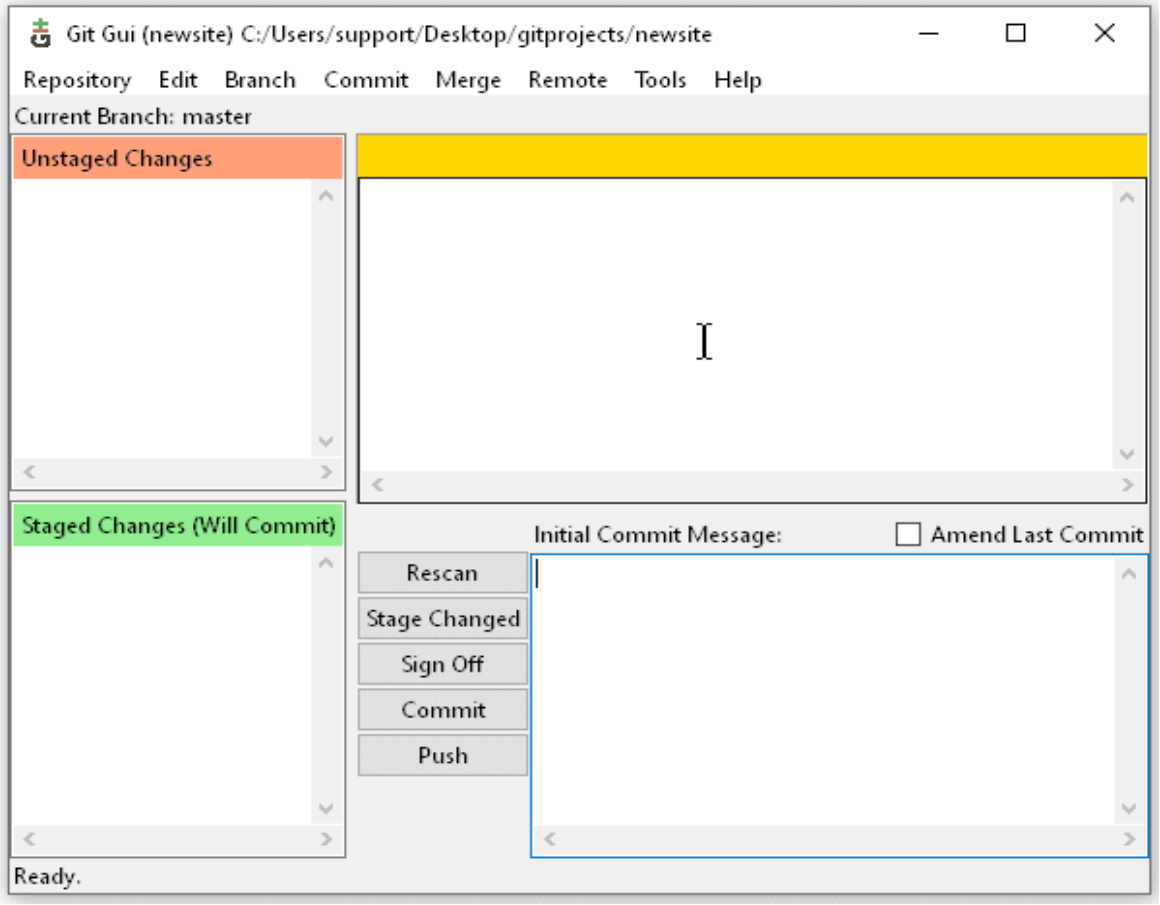
Основни Git понятия и команди

Другите две опции са: ***Clone Existing Repository*** – клониране на съществуващо хранилище и ***Open Existing Repository*** – отваряне на вече създадено.
Изберете с ***Browse*** локална директория, в която ще се създаде Git хранилище.
Кликнете ***Create***. Избор на директория, в която ще се намира Git хранилището.
Интерфейсът за работа с Git ще се отвори.



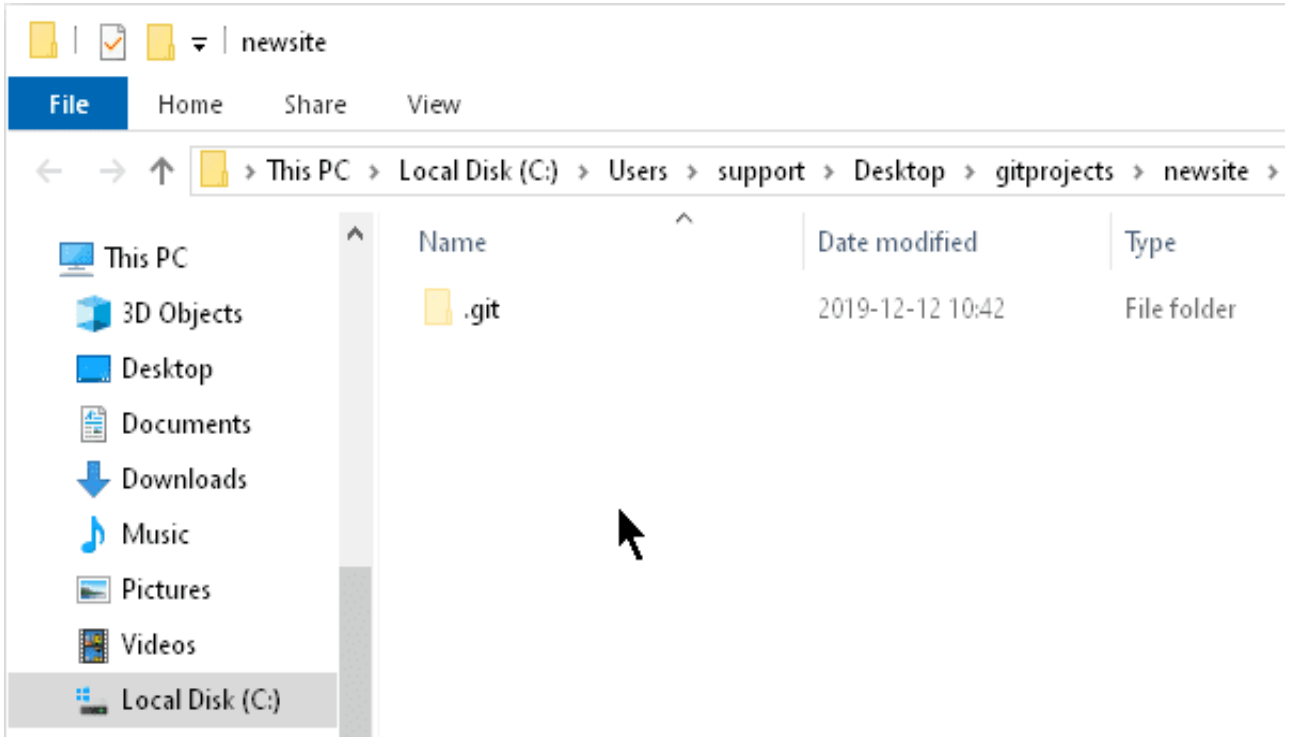
Основни Git понятия и команди

(GUI) интерфейс за работа с Git.



Основни Git понятия и команди

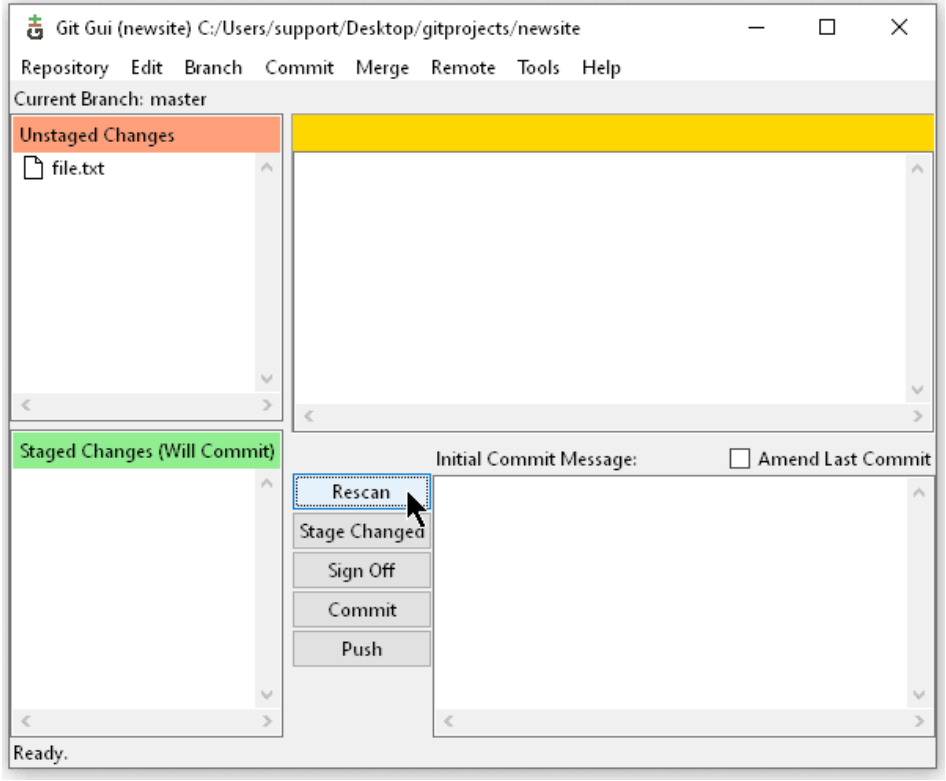
Отворете директорията на проекта, като изберете от главното меню Repository » Explore Working Copy.



Основни Git понятия и команди

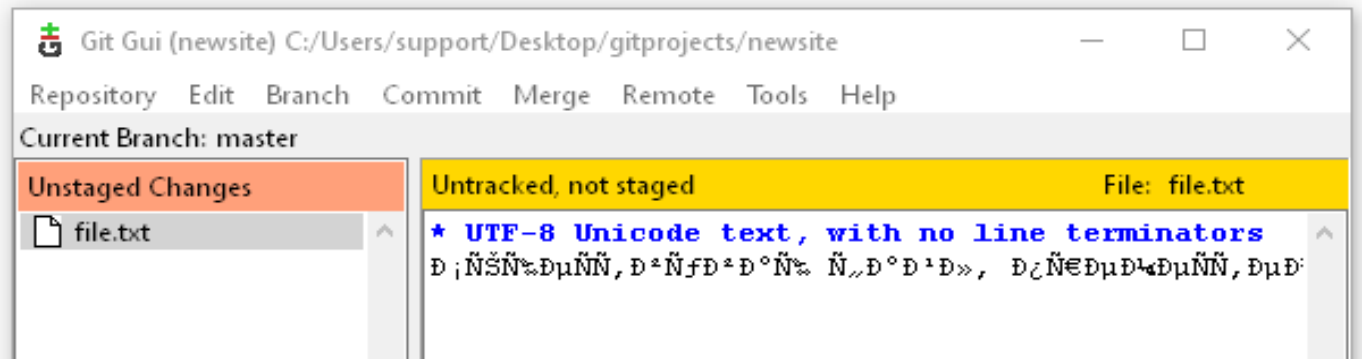
Директорията на Git хранилището.

Към момента хранилището е празно. Добавете един или повече файлове в него. Ако файлът съществува вече и само го преместите/копирате в тази директория, трябва да кажете на Git да го следи за промени. След като добавите файл, сканирайте за промени в директорията с Rescan в Git клиента. Git засича промените в директорията на хранилището, открива нов файл, който все още не следи. Засечените промени ще се покажат в раздел **Unstaged Changes**.



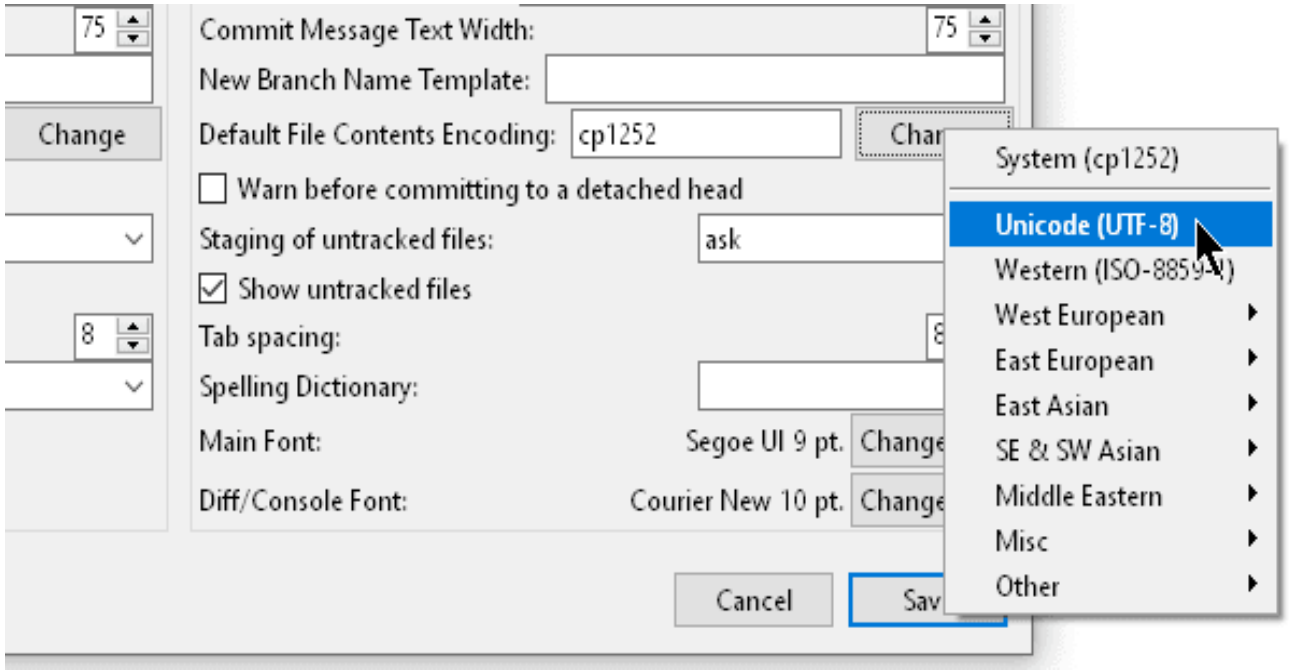
Основни Git понятия и команди

Ако съдържанието на файла се показва като неразбираеми символи, може да зададете Git клиентът да използва UTF-8 енкодинг.
Съдържанието на файла е в енкодинг UTF-8, а Git го визуализира в cp1252.



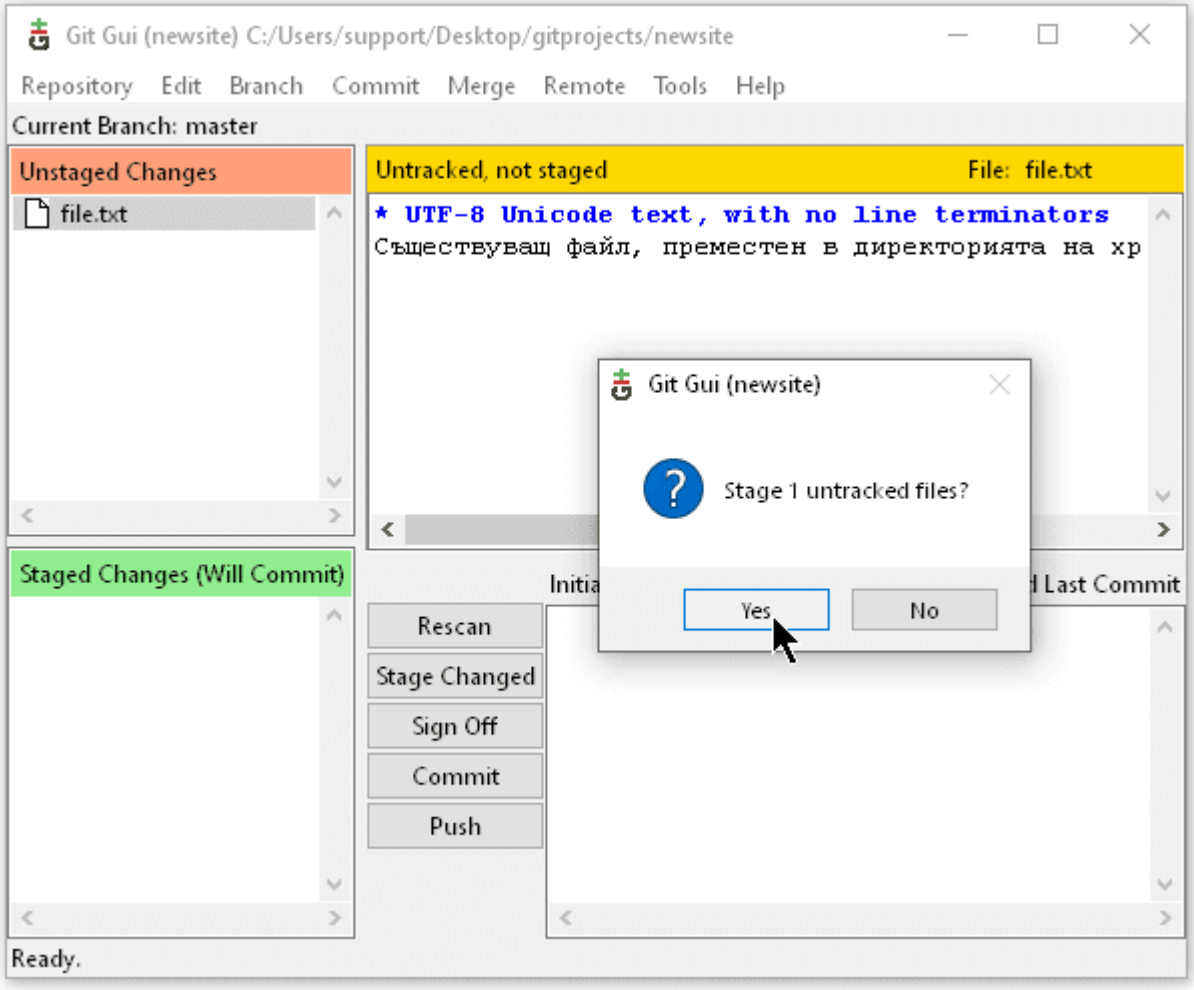
Основни Git понятия и команди

През Edit » Options » Default File Contents Encoding (Global) – кликвате на бутона Change и избирате от списъка Unicode (UTF-8), записвате промените със Save. Енкодинг по подразбиране за съдържанието на файловете. Рестартирайте клиента, за да влязат в сила промените на енкодинга.



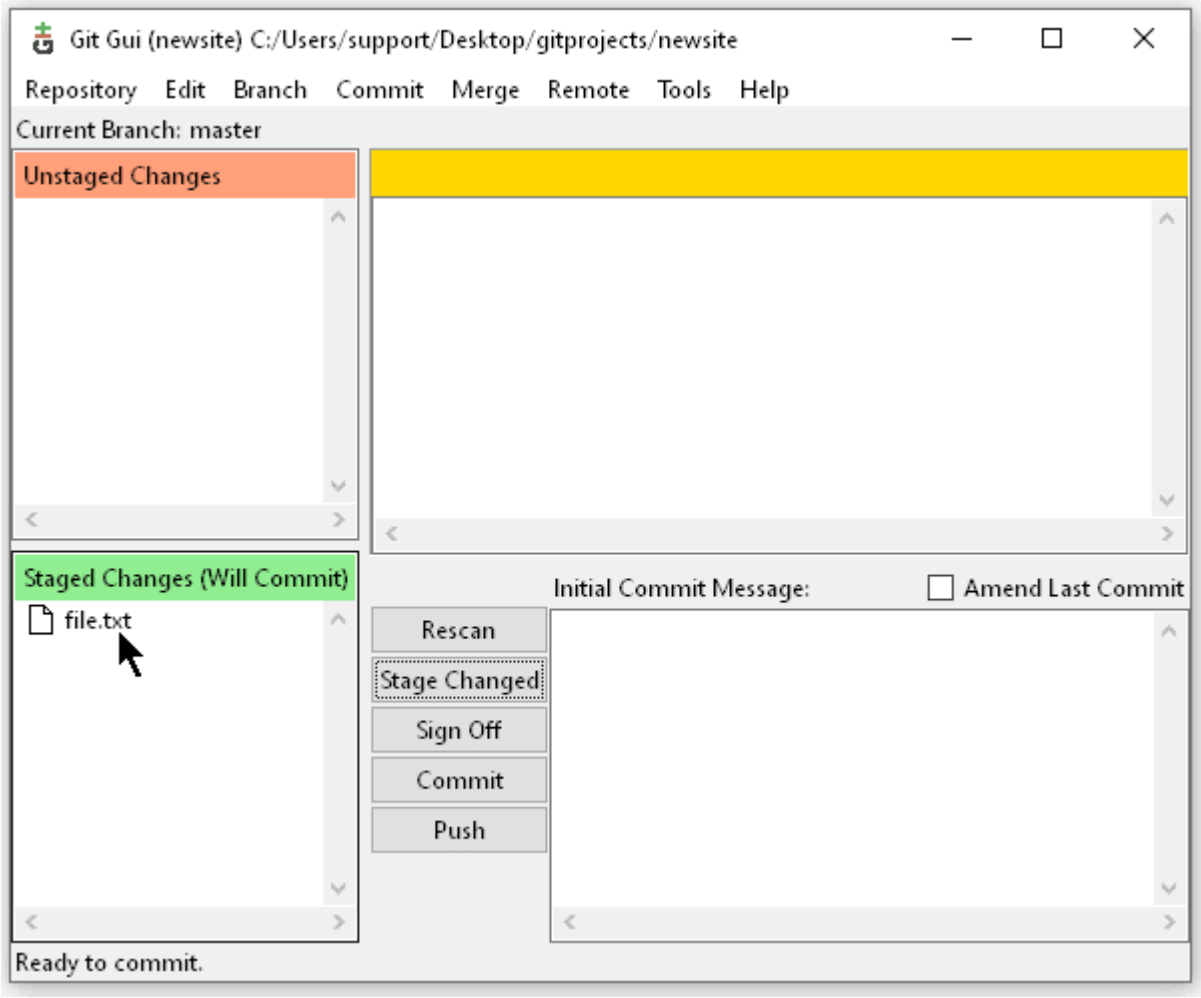
Основни Git понятия и команди

Git клиентът показва коректно съдържанието на файла. Подготовка за публикуване на промените в него (stage). Подгответе промените за публикуване, като кликнете на Stage Changed и изберете Yes на съобщението (дали да добави файла, който към момента не се следи (untracked)).



Основни Git понятия и команди

Staged Changes – промени, готови за публикуване.



Основни Git понятия и команди

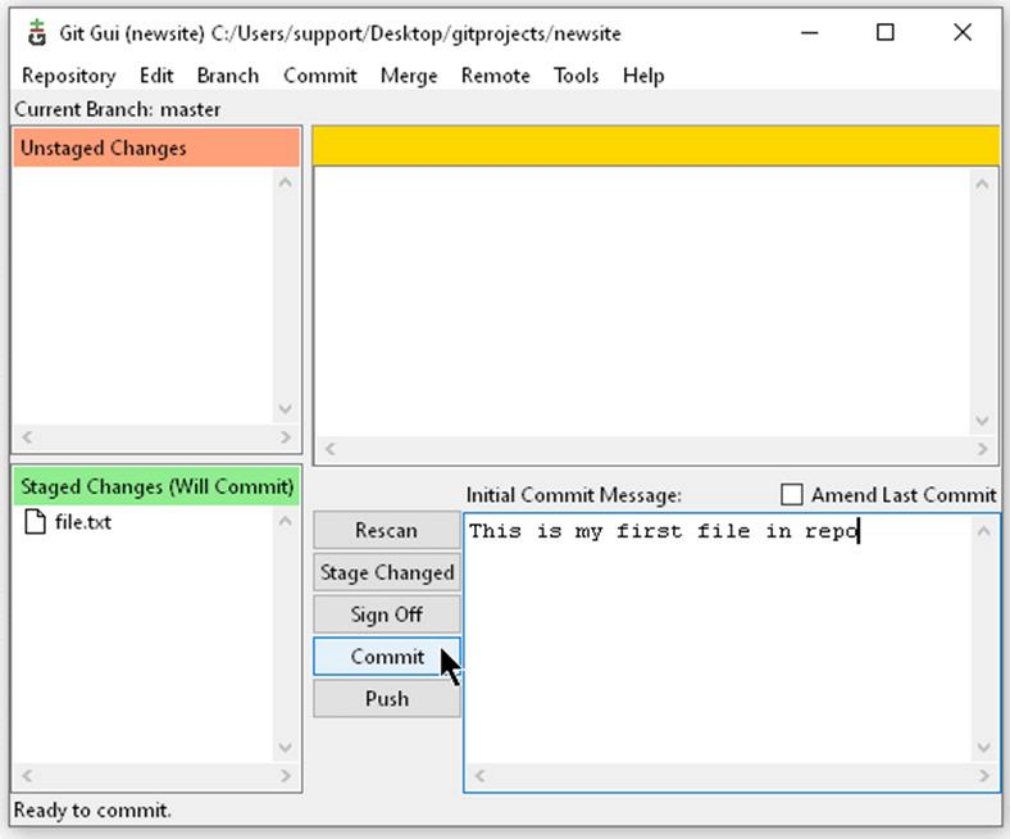
Staged Changes е статус, в който се поставя файла, или може да се разглежда като временен виртуален контейнер, в който се задържат промените, които са готови да бъдат публикувани (**commit**). Можете да събирате промени, да връщате някои от тях в етап на разработка и след това отново да ги добавяте в този статус. При много промени, можете да изберете само определени, които ще бъдат публикувани.

Всички промени в статус **Staged** се преглеждат и ако са одобрени, може да се публикуват.

За да публикувате промените в хранилището (и да се добави новият файл към системата за следене на промените **Git**), попълнете кратко описание на промените и кликнете на **Commit**.

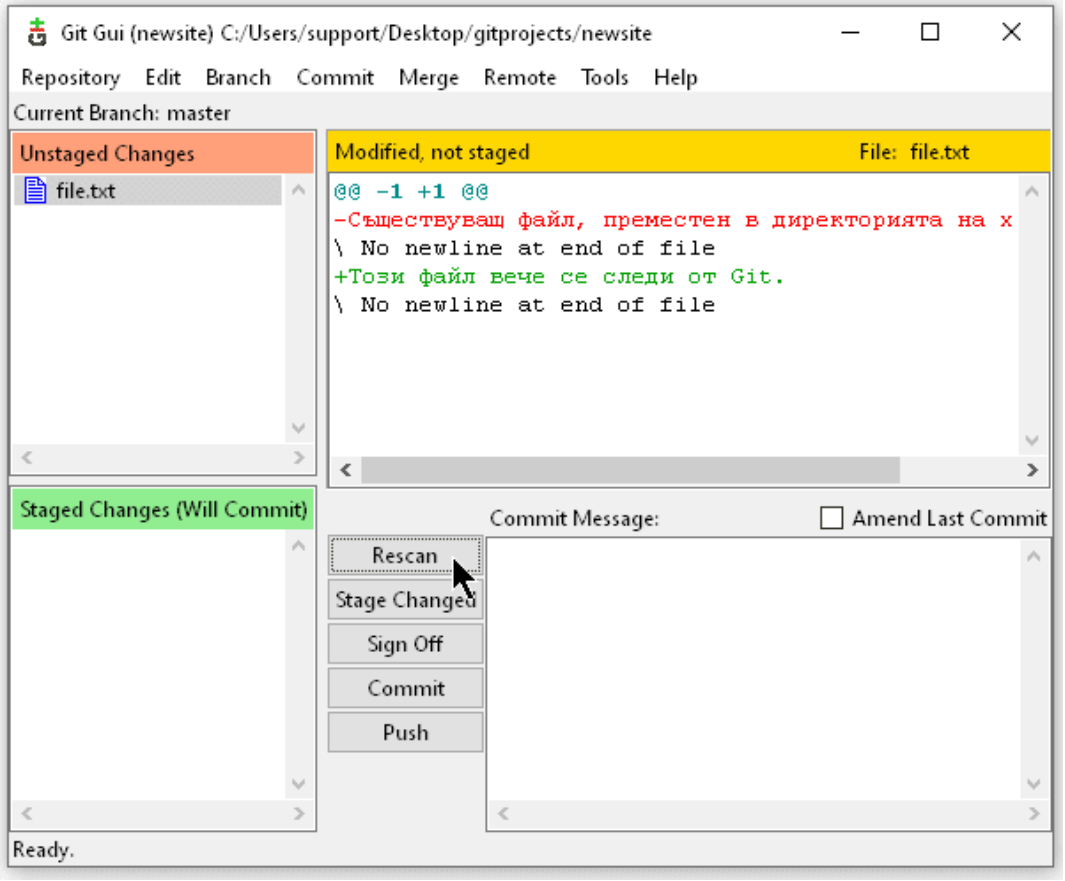
Основни Git понятия и команди

Commit – публикуване на промените. Файлът се премахва от раздел Staged Changes и Git вече го следи за промени.



Основни Git понятия и команди

Може да отворите файла в текстов редактор, да промените нещо в него и да запишете промените. След това в Git клиента кликнете на **Rescan** и ще видите промените в раздел **Unstaged Changes** – очакващи проверка и подготвяне за публикуване (преместване в **Staged Changes**). Извършени промени във файла.



Основни Git понятия и команди

Създаван е на ново локално Git хранилище през Git Bash Същите действия за създаване и управление на Git хранилище можете да извършите с изпълнение на команди през Git Bash. Създаване на ново хранилище и добавяне на файл в него през Git Bash.

MINGW64:/c/Users/support/Desktop/gitprojects/newsiterepo

```
support@support MINGW64 ~/Desktop/gitprojects
$ cd newsiterepo

support@support MINGW64 ~/Desktop/gitprojects/newsiterepo
$ git init
Initialized empty Git repository in C:/Users/support/Desktop/gitprojects/newsiterep
o/.git/

support@support MINGW64 ~/Desktop/gitprojects/newsiterepo (master)
$ echo "contents of file" >> newfile.txt

support@support MINGW64 ~/Desktop/gitprojects/newsiterepo (master)
$ git add .
warning: LF will be replaced by CRLF in newfile.txt.
The file will have its original line endings in your working directory

support@support MINGW64 ~/Desktop/gitprojects/newsiterepo (master)
$ git commit -m "New file added"
[master (root-commit) 9efc8e8] New file added
 1 file changed, 1 insertion(+)
 create mode 100644 newfile.txt

support@support MINGW64 ~/Desktop/gitprojects/newsiterepo (master)
$
```

Основни Git понятия и команди

За да създадете ново празно хранилище, заредете Git Bash, влезте в директорията, която ще се използва за хранилището и изпълнете командата:

git init

Тази команда инициализира Git хранилище и създава подпапката .git.

Можете да създадете нов файл в хранилището с команда:

echo "contents of file" >> newfile.txt

Нови файлове в хранилището можете да добавяте по всякакъв начин, например да ги премествате в директорията или да ги създадете чрез команди за създаване на файлове (например touch).

Подгответе промените за публикуване (поставяне в статус Staged) чрез командата:

git add newfile.txt

Публикувайте промените в локалното хранилище:

git commit -m "New file added"

След като имате Git хранилище локално на Вашето устройство, можете да се свържете с отдалечено хранилище, в което да публикувате промените. Например отдалеченото хранилище може да се намира във Вашия хостинг акаунт (създадено през cPanel – Git Version Control). Възможно и работа с командите на Git от cPanel-а.

Github

Основни понятия и команди в GitHub

Github е много известна **платформа за съхранение, разпространение и управление на отворен код**. Github се използва от много разработчици по целия свят, сред които има големи компании като Microsoft, RedHat и други.

Github предоставя не само възможността да преглеждате кода и да го разпространявате, но и историята на версиите, инструментите за съвместна разработка, инструментите за предоставяне на документация, издаване на издания и обратна връзка. И както в Git можете да публикувате както отворени, така и частни проекти в Github. Тук ще разгледаме като за начинаещи как да използваме Github за хостинг на вашия проект.

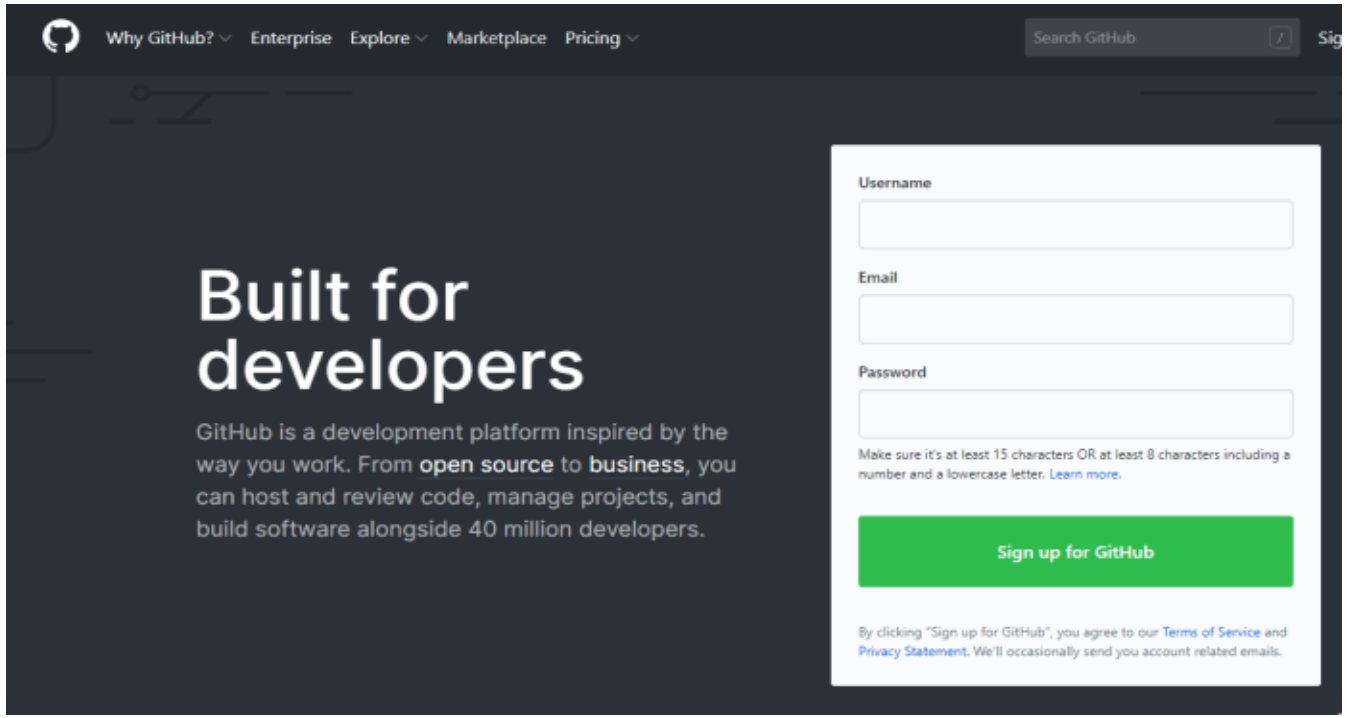
Издания на Github

GitHub Issues е една от най-популярните системи за проследяване на грешки в света. Той предоставя на собствениците на хранилища възможността да организират, маркират и обвързват проблеми с контролни точки. Ако откриете проблем в проект, управляван от някой друг, той ще бъде отворен, докато не го затворите (например, ако разберете какъв е проблемът) или докато собственикът на хранилището не го затвори. Понякога ще получите окончателен отговор, а понякога проблемът ще остане отворен и ще бъде маркиран с някаква информация, която го класифицира. След това програмистът може да се върне към маркера, за да отстрани проблема или да подобри кодовата база, използвайки вашата обратна връзка.

Повечето разработчици могат безплатно (виж по-нататък) да поддържат своя код, публикуван в GitHub, така че не могат да се очакват бързи отговори. Но някои репозитарии с отворен код се публикуват от компании, които предоставят услуги за този код. Те предлагат комерсиални предложения за версии с много функции или използват архитектура на приставки. Следователно те плащат на разработчиците, работещи по проект с отворен код.

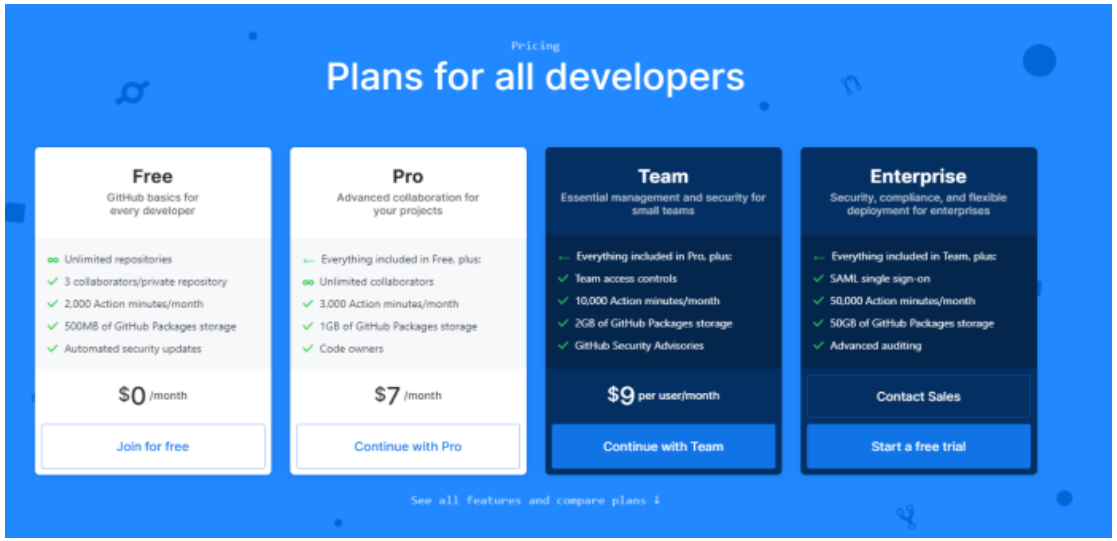
Създаване на акаунт в Github

За да създадете нов акаунт на сайта, отворете главната страница на GitHub и веднага можете да въведете данни за новия акаунт. Трябва да посочите потребителско име, имейл и парола. Не ви е необходима никаква настройка освен няколко клика на мишката.



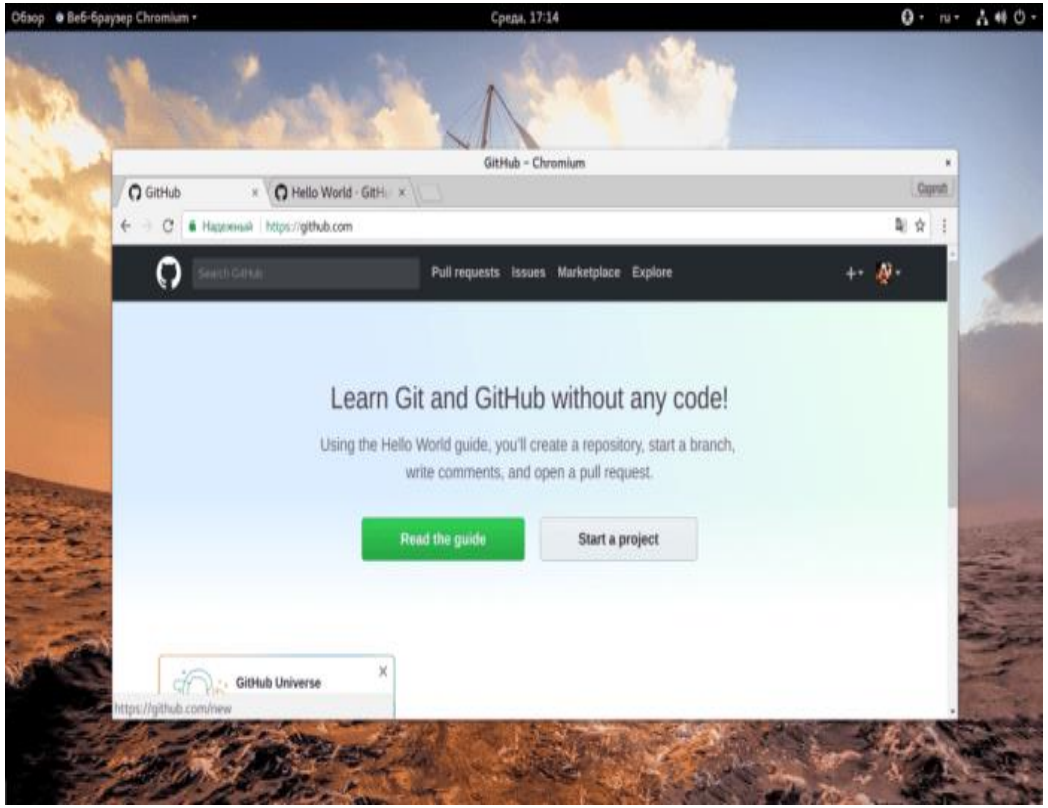
Избор на вида на хранилището и тарифен план

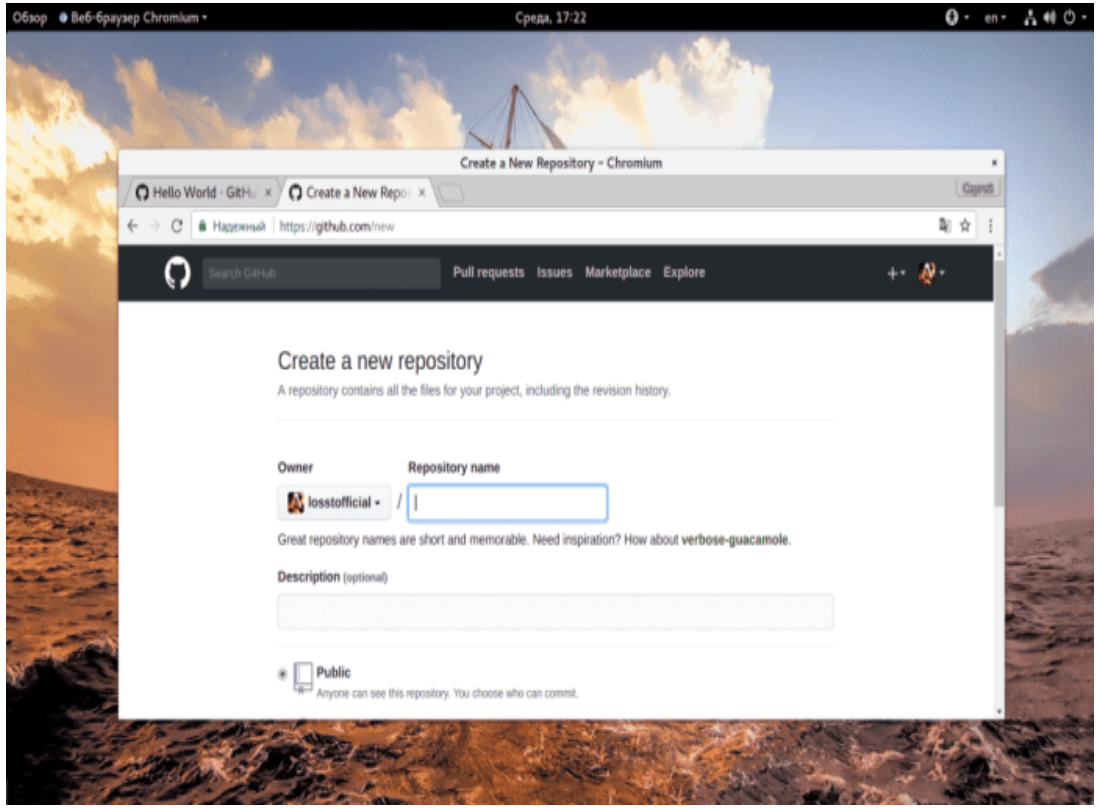
В следващата стъпка трябва да изберете типа хранилище. За проекти с отворен код (open-souce) ползването на сайта е безплатно. Ако трябва да имате частни хранилища (репозитори), е възможно да преминете към платен тарифен план. Профилът е готов и ще бъдете пренасочени към страница, където можете да създадете първия си проект. Но преди да можете да направите това, трябва да потвърдите своя имейл адрес. За целта отворете входящата си поща и следвайте връзката в писмото от Github. Сега нямаме нито едно хранилище и можем или да създадем ново хранилище или да разклоним (fork) от съществуващо чуждо хранилище и да поддържаме собствен клон за развитие. След това, ако желаете, можете да предложите промените на автора на оригиналното хранилище (Pull request- изтеглете заявка).



Създаване на хранилище в Github

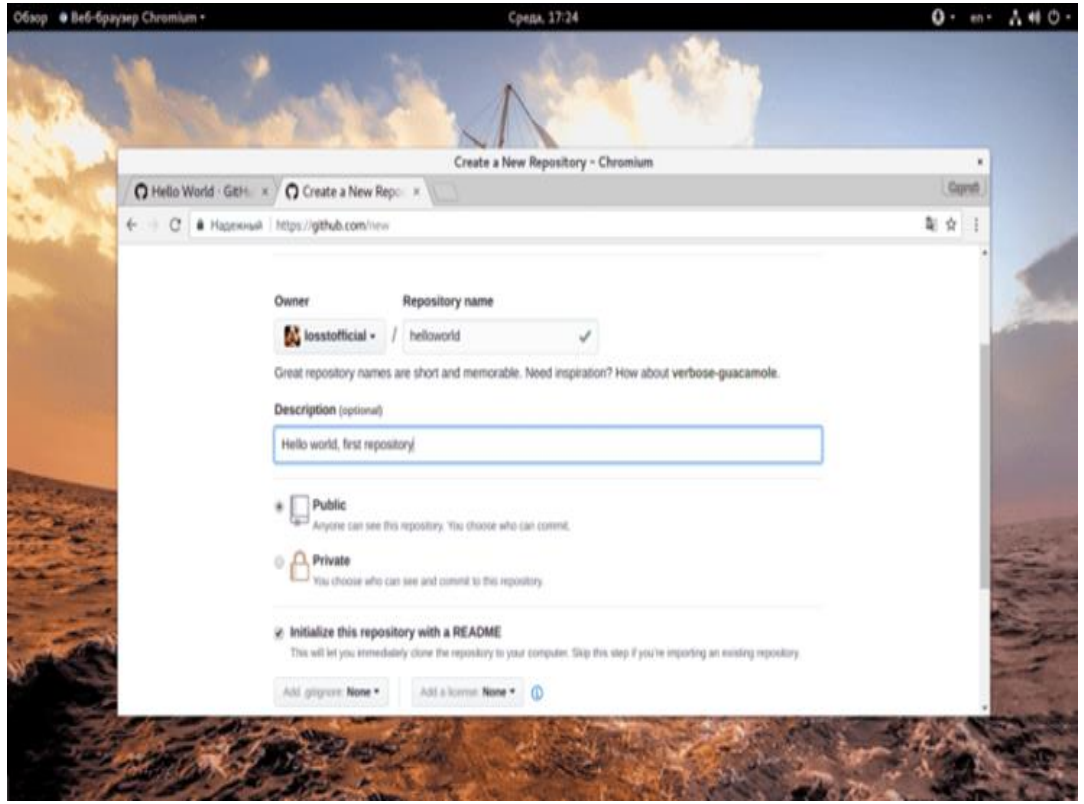
На страницата, която се отваря, това е основната страница за оторизирани потребители, кликнете върху бутона „Стартиране на проект“ и след това въведете името и описанието на бъдещото хранилище.

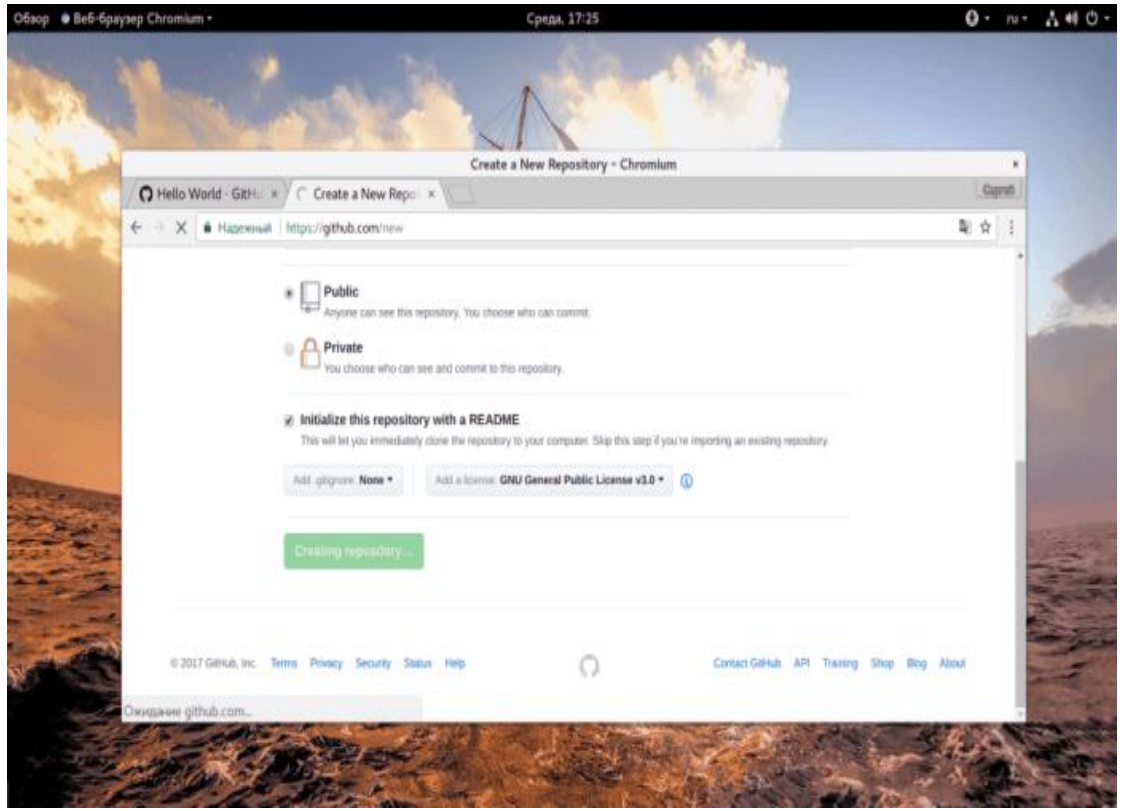




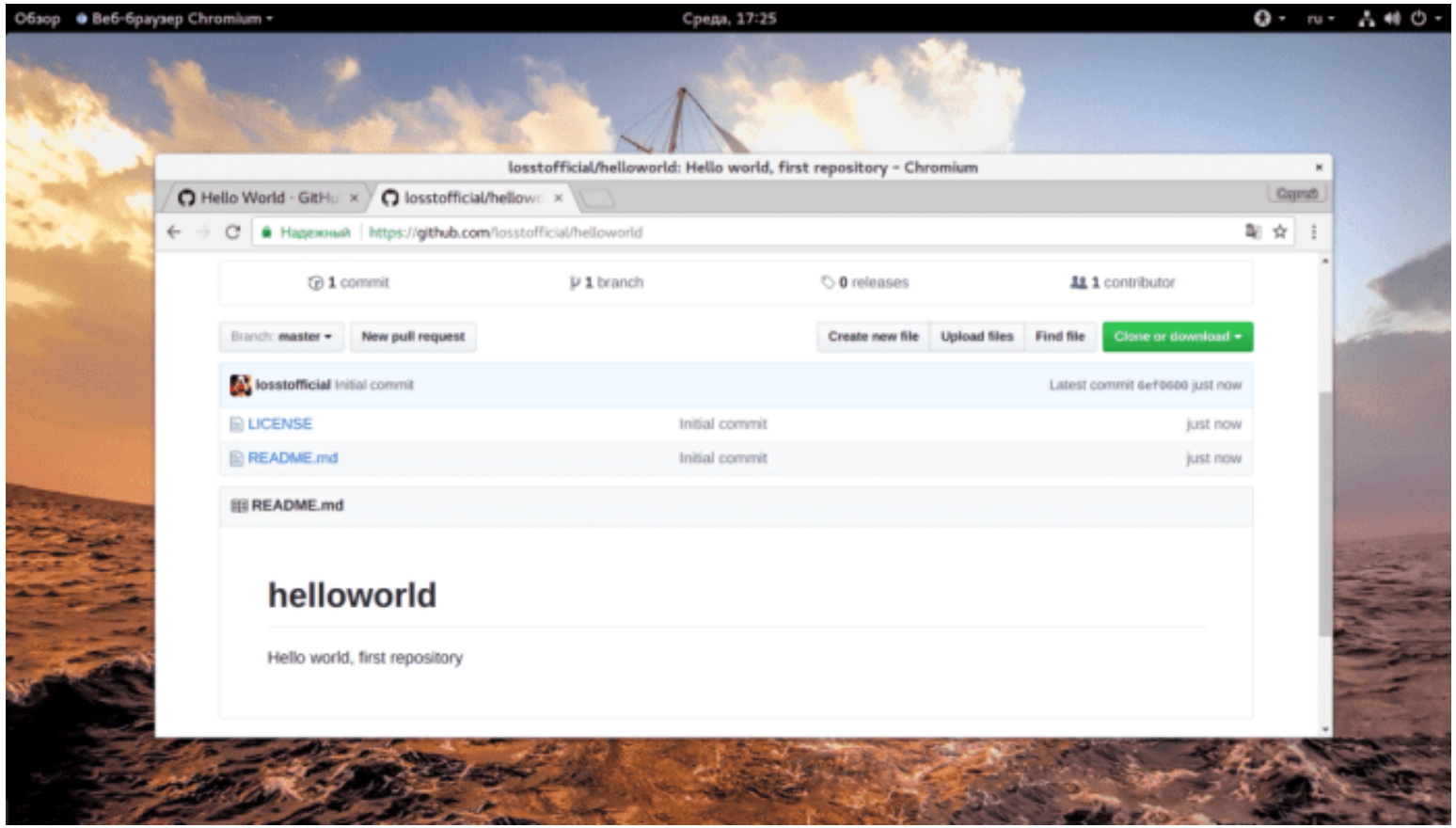
Основни GitHub понятия и команди

Можете веднага да инициализирате хранилището, като създадете файл Readme. За да направите това, поставете отметка в квадратчето „Initialize this repository with a README“ в долната част на страницата. Можете също да изберете лиценз. Когато всчико е готово, изберете ”Create project” ще бъде създаден нов проект с файл README, в който се намира описание и файл с лиценза.





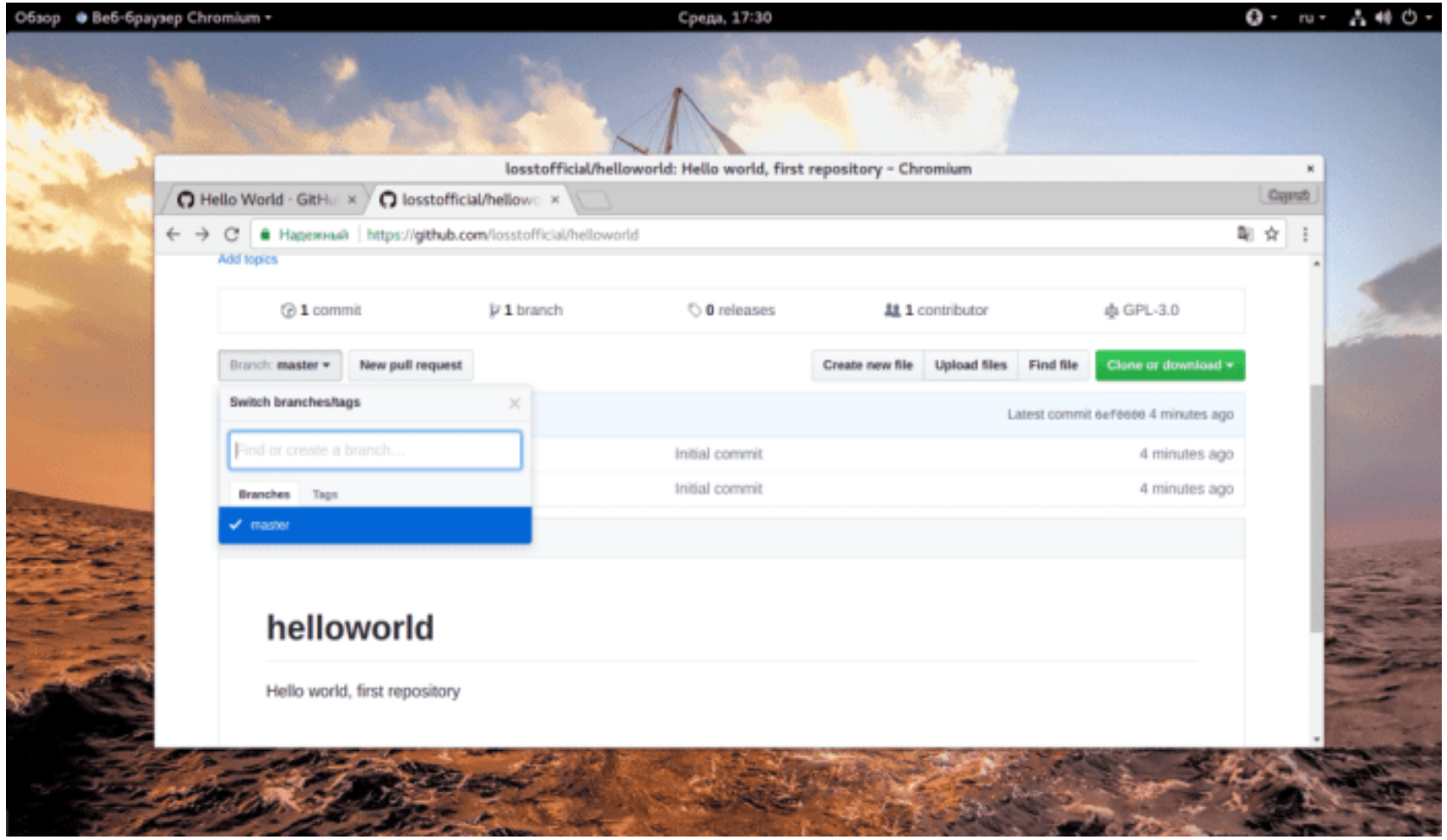
Файл с лиценза на новото хранилище



Добавяне на клонове

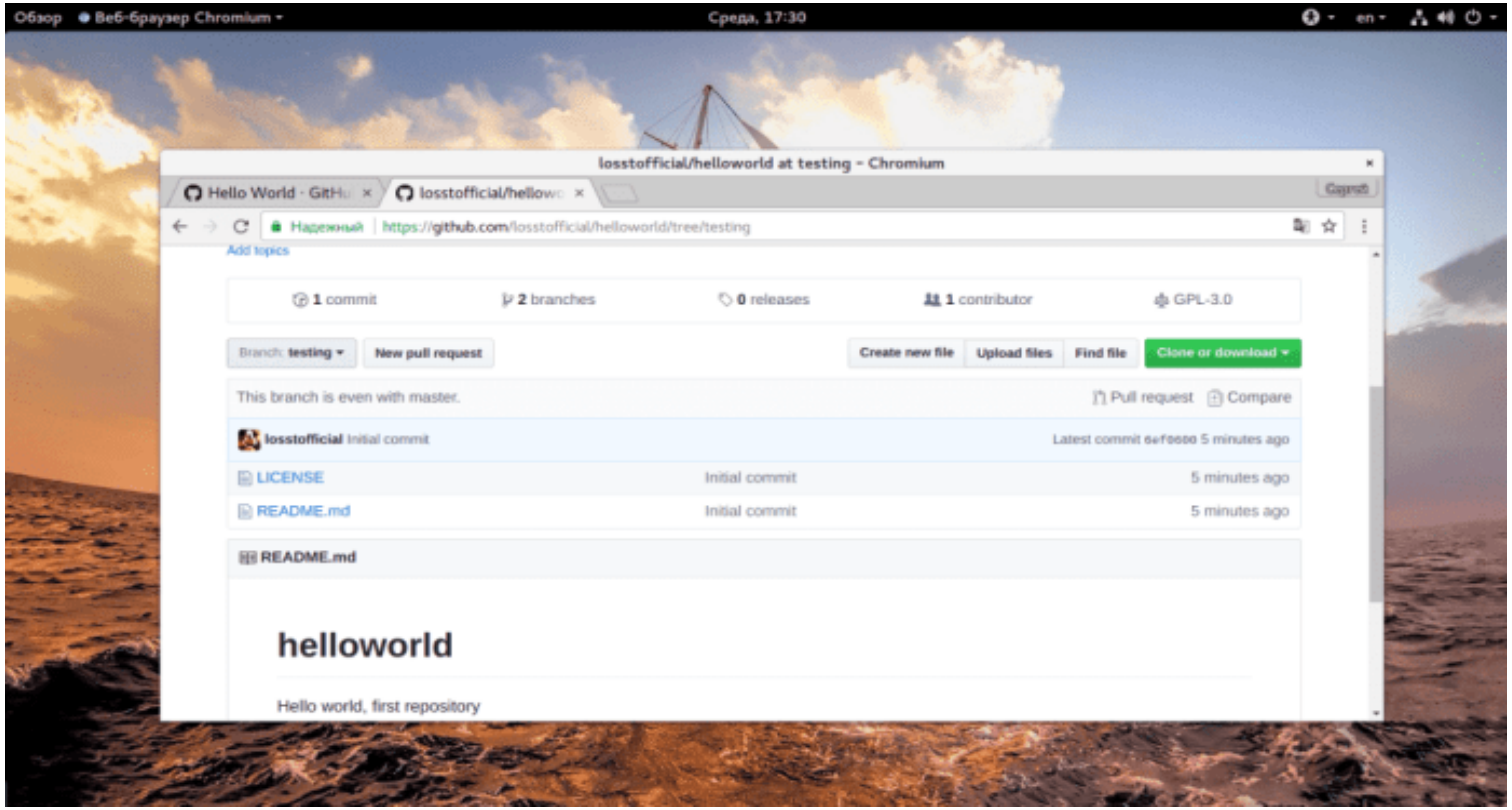
Клоновете на Github ви позволяват да работите с няколко версии на проект едновременно. По подразбиране при създаване на хранилище се създава **главния клон**, това е основният работещ клон. Можете например да създадете допълнителни клонове, за да тествате софтуера преди публикуването му в основния клон. По този начин можете едновременно да разработите продукт и да предоставите на потребителите стабилна версия. Можете също да създадете отделни разклонения за версията на програмата за различни системи.

Създаване на клон на проекта



Добавяне на клонове

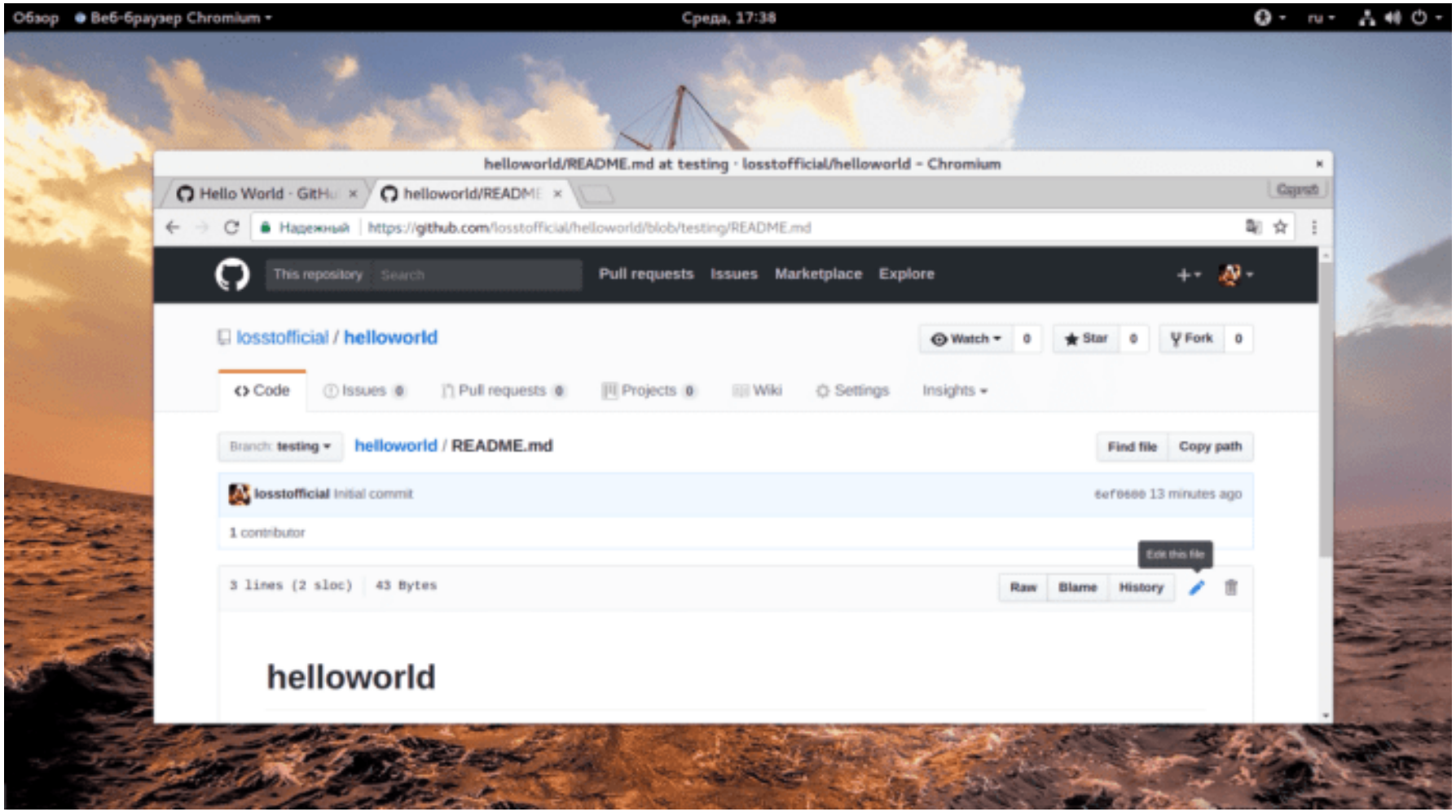
Текущият клон е посочен в горния ляв ъгъл след думата „Branch“. За да създадете нов клон, просто разширете този списък и започнете да пишете името му. Самият сайт ще ви предложи да добавите нов клон, изберете „Създаване на клон“. Веднага след създаването, ще работите с новосъздадения клон.



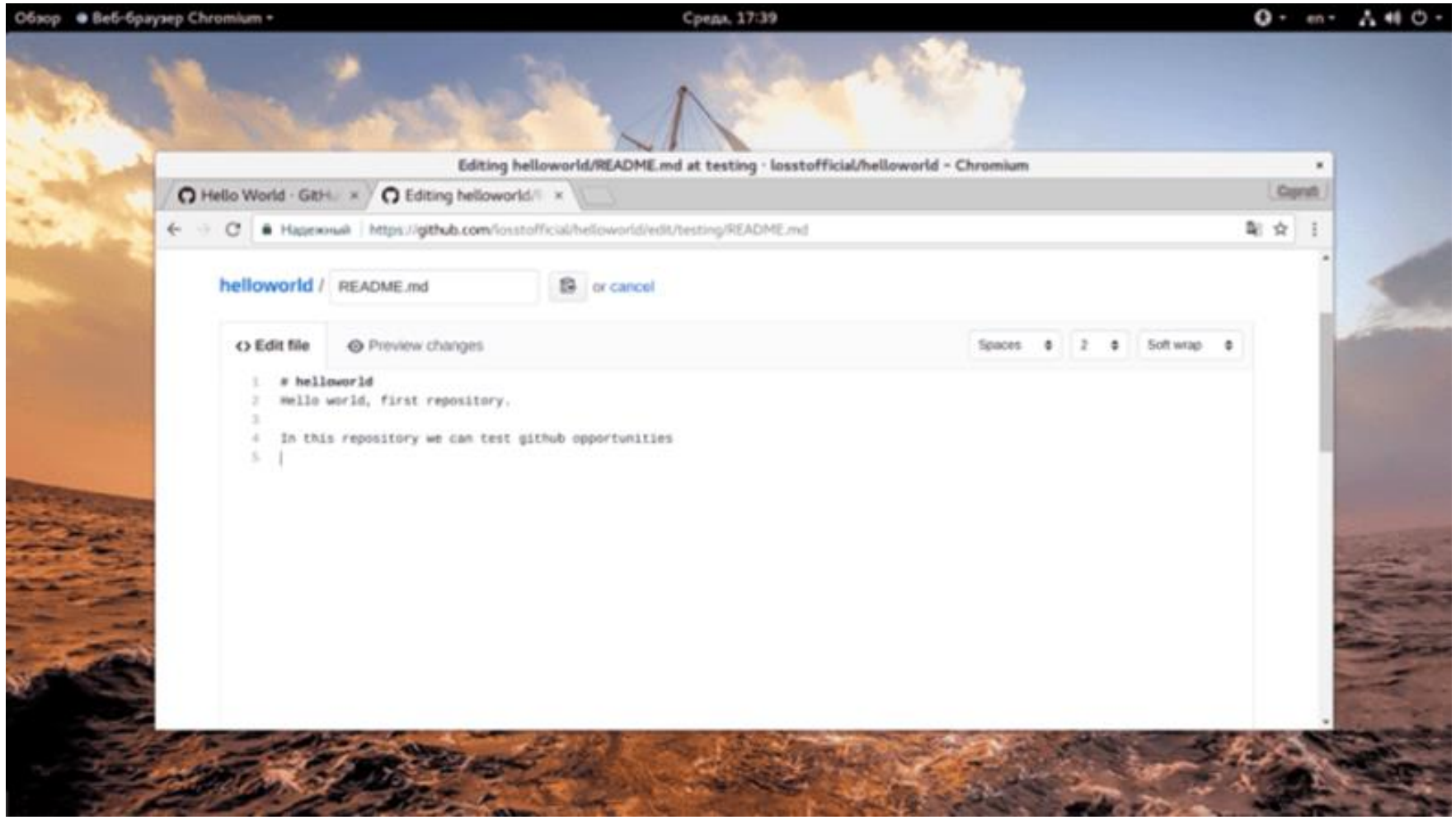
Промяна на файлове и комити

Всички промени във файловете на Github се правят с помощта на комити. Поема се ангажимент, като се правят самите корекции и се опишат тези корекции. Това е необходимо, за да знаете какво и кога сте се променили, а също така лесно да проследявате работата на екипа. Думата ангажиране може да се преведе като „ангажиране“. Тоест, можем да направим промени в няколко файла и след това да ги ангажираме. Нека например променим файла README. За целта намерете бутона с четка в дясната страна на панела и кликнете върху него:

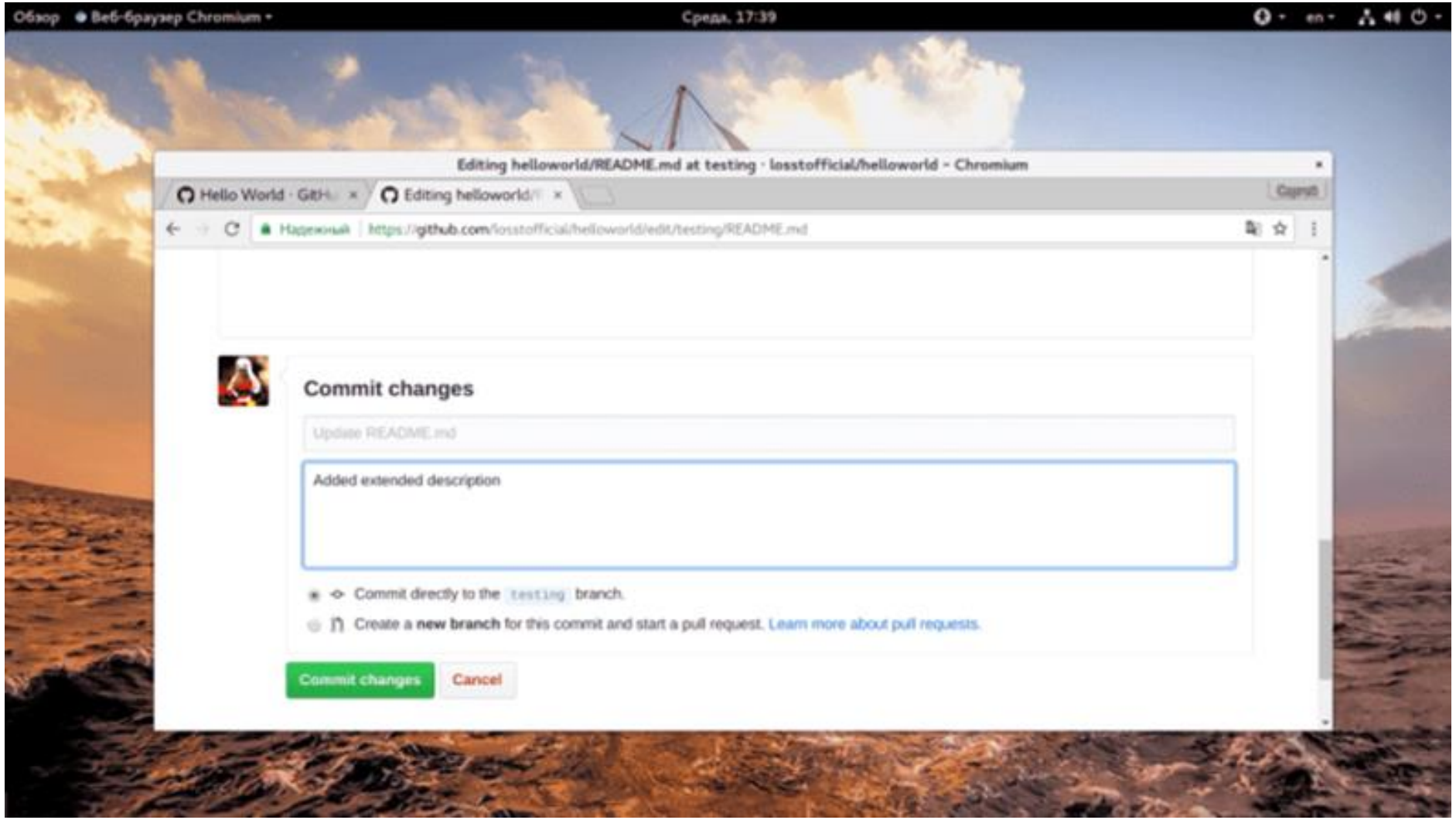
Създаване на комит



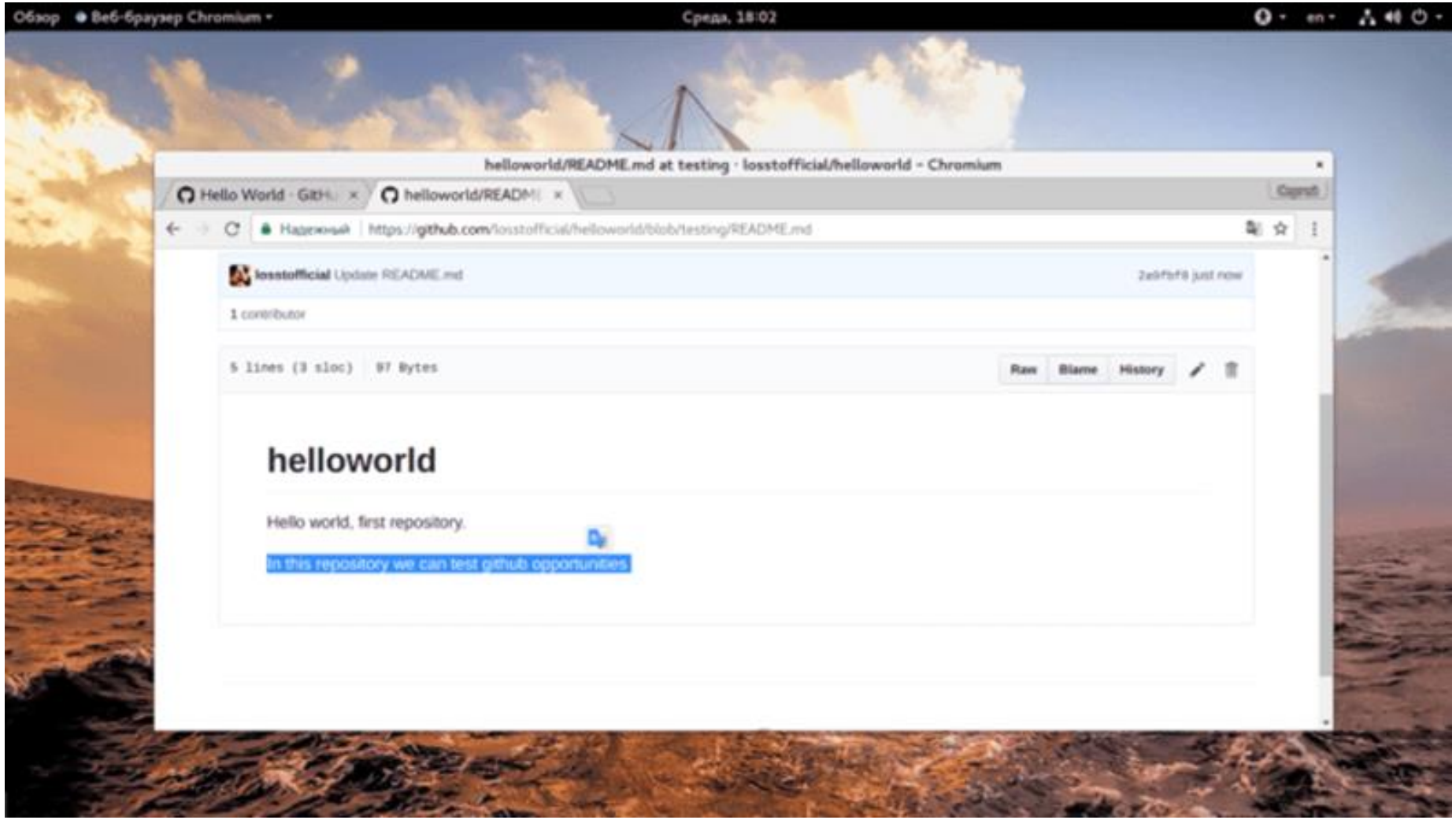
Текстов редактор за промяна на файла



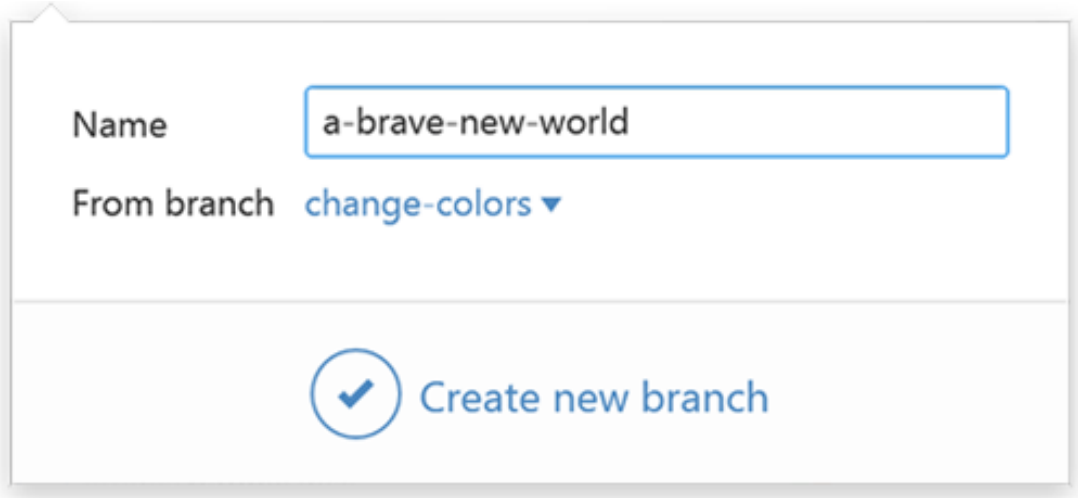
Запазване на промените в Commit.



Промени в текущия клон на проекта

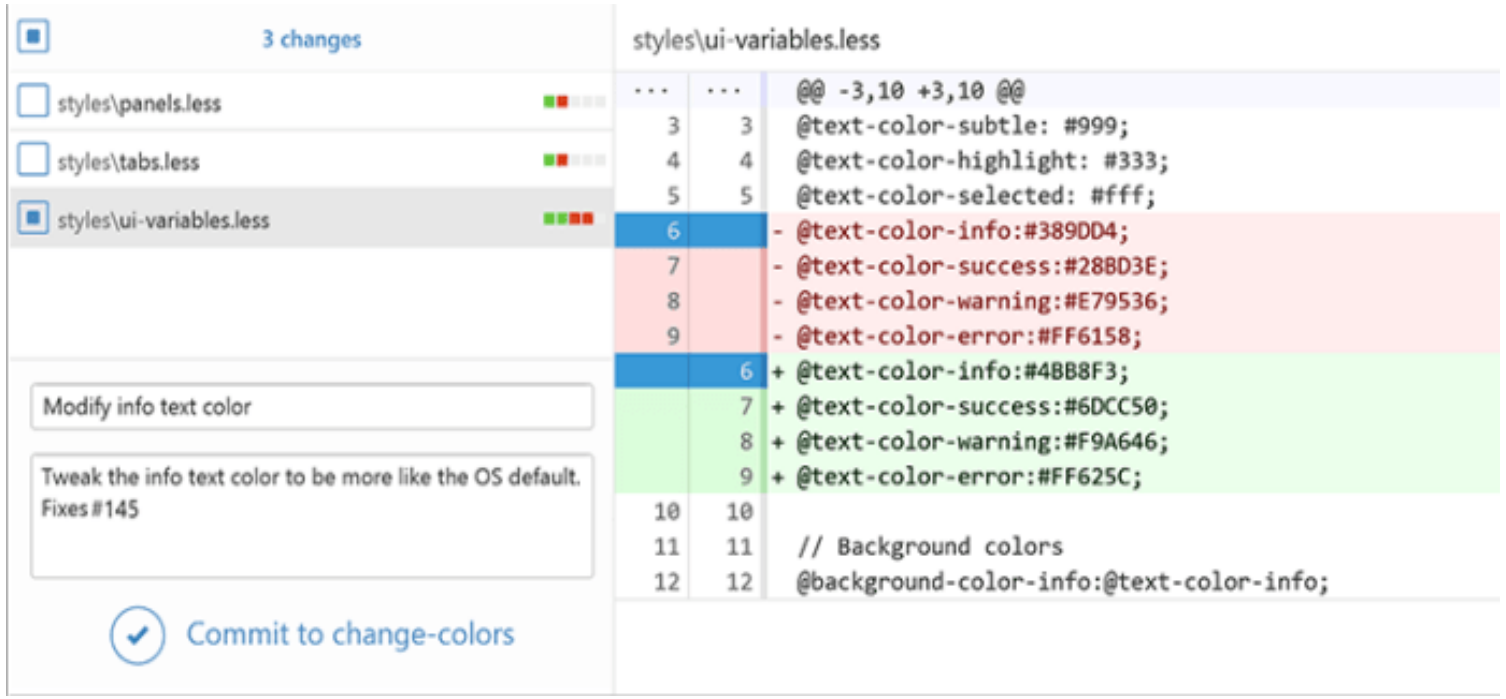


Github Desktop - Създаване на разклонения

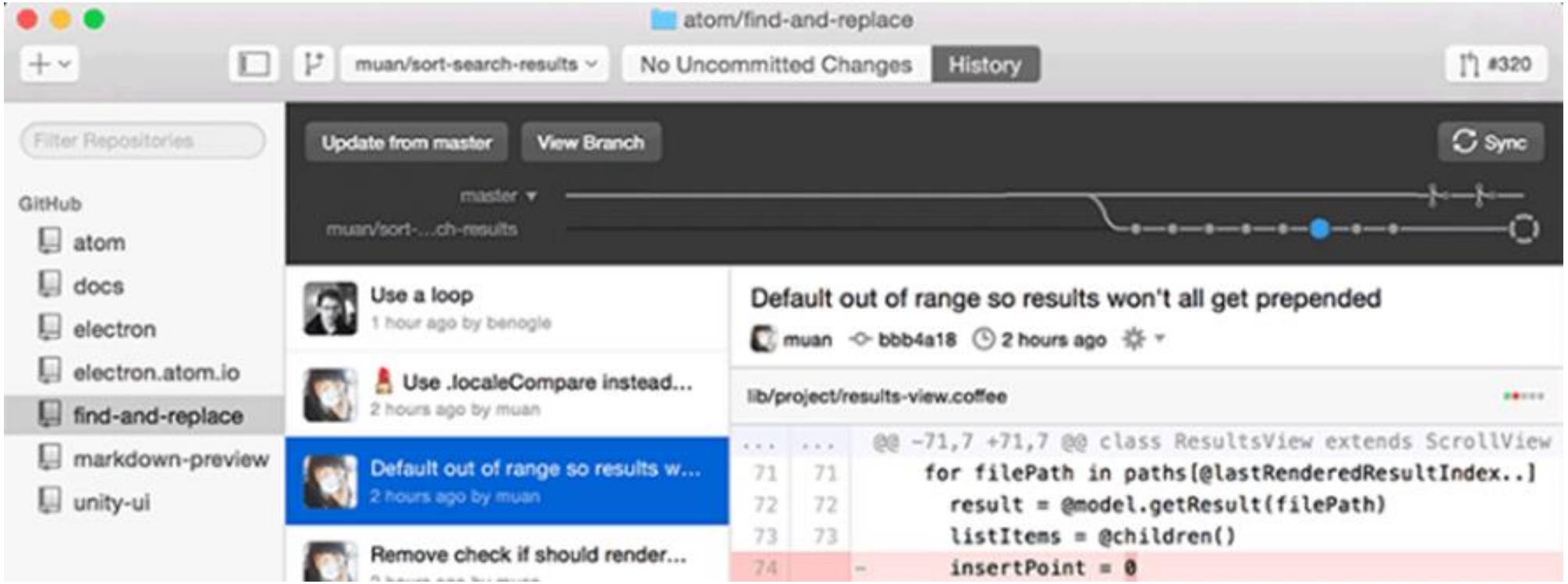


Съвместна работа

Преглеждайки промените (diff) преди да изпратите комита на сайта, програмата веднага показва в кои файлове и редове са направени промените. Комитът се изпраща от прозореца на програмата, без да се използва командния ред. Преглед на промените (на фигурата)



Обединяване и внедряване



Преглед на историята

Налична е интерактивна графика с визуализация на направените промени. Отдясно на диаграмата можете да изберете ангажимент и да видите историята на промените в местния клон.

Команден ред на Github

Според опита най-добрата загуба на време за овладяването на Github чрез командния ред при работа с технологии с отворен код. Да, има много добри графични интерфейси, но всички те са по-малко гъвкави за използване. Освен това има инструменти само за командния ред, които значително опростяват живота и увеличават ефективността на развитието:

• **hub** - обвивка за Git, която улеснява работата. Няма значение дали сте начинаещ или опитен разработчик с отворен код, хъбът ще улесни извличането на хранилища, навигирането в страниците на проекти, работата с вилици и дори изпращане на заявки за изтегляне. И всичко това от командния ред. hub.github.com

• **tj / git-extras -** набор от помощни програми на Git, като обобщение на хранилището, **repl**, дневник на промените, статистика на комитите от автори и много други. **github.com/tj/git-extras**

Управление на проекти

Наред с проблемите, благодарение на които разработчиците получават обратна връзка от потребителите, интерфейсът предлага и други функции, които ви позволяват да управлявате проекти.

Един от тях са Проекти и това е системата Kanban, която помага да се организират търсене на бъгове и работата, която трябва да се свърши по проекта.

Има Wiki, което е предназначено да се използва като документация за потребителите. Доста впечатляващо е програмният език GitHub Wiki Go.

Иползват се контролни точки които помагат при управлението на проекти. Това е част от страницата със задачи. Можете да свържете задачаите с конкретни точки на прекъсване, които може да са следите целите на изданията. Представяйки изданията, GitHub разшири функционалността на GIT тагове. Маркерът GIT е указател към конкретна версия. Ако той се изпълнява последователно, той ви помага да се върнете към предишната версия на кода, без да се позовавате на конкретни версии.

Управление на проекти

Изданието е изградено на базата на GIT тагове и представлява пълната версия на вашия код, както и ZIP файлове, бележки за освобождаване и двоични ресурси, които могат да представят напълно работеща версия на крайния код продукт.

Въпреки че можете да създадете GIT маркер програмно (например, използвайки git тага от командния ред), създаването на GitHub версии е ръчен процес, който се осъществява в потребителския интерфейс на GitHub.

По същество казвате на GitHub да създаде ново издание и ви уведомява към кой маркер искате да го приложите.

Webhooks и Services на GitHub

GitHub предоставя много функции, които подпомагат работния процес на разработчиците: например, уеб кукита и други услуги.

• **Webhooks**

Webhooks ви позволяват да пингва външни услуги, когато се случват определени събития. Например, това може да се случи, когато командата **push** се използва за код, се създава клон или ако маркерът е създаден или изтрит.

Когато се случи събитие, GitHub изпраща **POST** заявка до URL адреса, който ние му казваме да използва.

Обикновено тази функция се използва за проверка на комуникацията с отдалечен сървър. Това е необходимо, за да получите най-новия код от GitHub, когато изпратим актуализацията от нашия локален компютър.

Изпращаме **push** командата до GitHub, тя казва на сървъра за това и сървърът извлича данните.

Webhooks и Services на GitHub

- **Services**

Услугите и новите приложения на GitHub са интеграции на трети страни, които подобряват опита на разработчиците или предоставят услуги.

Например, можете да настроите изпълнител на тестове, за да стартира тестове автоматично всеки път, когато правите нови ангажименти.

TravisCI ще помогне тук.

Можете да конфигурирате непрекъсната интеграция с помощта на **CircleCI**.

Можете да създадете интеграция на Codeclimate, която анализира кода и предоставя отчет „техническия дълг“ при покриване на тестването.

Заключение

Системата за контрол на версиите е софтуер, който помага на разработчиците на софтуер да работят съвместно и да поддържат пълна история на своята работа. Той може да съхранява промени в файлове и модификации на изходния код. Всеки път, когато потребителят промени проекта, системата за контрол на версиите отчита състоянието на проекта и ги записва. Тези различни запазени състояния на проекта са известни като версии. Например, ако програмистът създава уебсайт, той се съхранява като версия 1. По-късно, ако програмистът добави друга страница към този уебсайт, тези промени се записват като версия 2. По същия начин промените се запазват като версии в системите за контрол на версиите. Git и Github са два термина, свързани с контрола на версиите. Ключовата разлика между Git и Github е, че Git е система за контрол на версиите с отворен код, а Github е хостинг услуга за хранилището на Git.

Благодаря за вниманието!

УСП - 3 НН